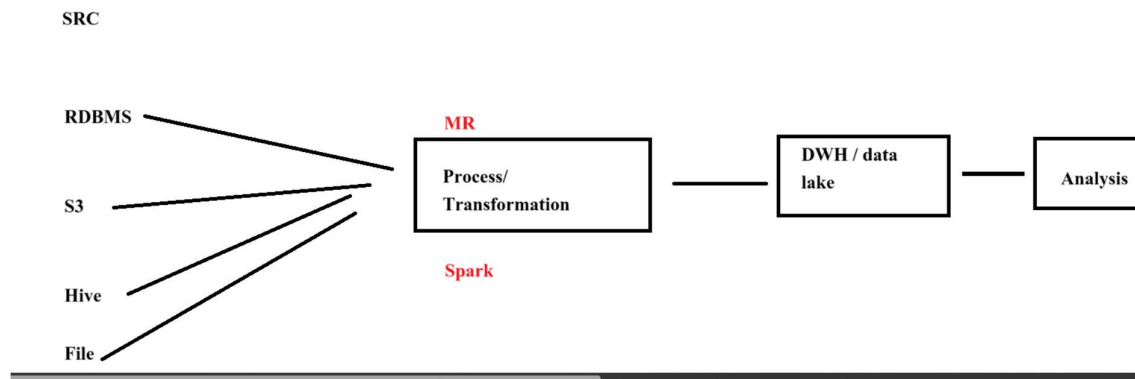




SPARK with Sayu Softtech

Spark Use case



Hadoop: Main Components

Storage : HDFS

Process : MapReduce

MapReduce :

processing Module

Computing Framework

Before of Spark there is Bigdata technologies Hadoop (HDFS and MR)

Sparks doing something more than Hadoop.

Disadvantages of MR

- 1.Speed
- 2.Process only Batch Data not for stream processing.
- 3.Not in memory Computation.
- 4.Process data stored on HDFS

Sparks:



- Apache Spark is an open-source distributed general-purpose cluster-computing framework. Spark provides an interface for programming entire clusters with implicit data parallelism and fault tolerance.
- It is Programming module
- Computing Framework
- work on distributed computing Module.
- Data process which is Distributed.
- By using programming module we can develop an application and run on distributed framework such as YARN
- 10 times better performance than MR.
- Spark has been written in SCALA language.

History of Spark :

Spark was initially started by Matei Zaharia at UC Berkeley's AMP Lab in 2009, and open sourced in 2010 under a BSD license.

In 2013, the project was donated to the Apache Software Foundation and switched its license to Apache 2.0. In February 2014, Spark became a Top-Level Apache Project.

In November 2014, Spark founder M. Zaharia's company Databricks set a new world record in large scale sorting using Spark.

Spark had in excess of 1000 contributors in 2015, making it one of the most active projects in the Apache Software Foundation and one of the most active open source big data projects.

Advantages of spark :

- 1.Provides highly reliable fast in memory computation.
- 2.Efficient in interactive queries and iterative algorithm.
- 3.Fault tolerance capabilities because of immutable primary abstraction named RDD.
- 4.Inbuilt machine learning libraries.
- 5.Provides processing platform for streaming data using spark streaming.
- 6.Highly efficient in real time analytics using spark streaming and spark sql.
- 7.Graphx libraries on top of spark core for graphical observations.
- 8.Compatibility with any API JAVA, SCALA, PYTHON, R makes programming easy.



Difference between Spark and MR

Table 1: Difference between MapReduce and Spark

<i>Hadoop MapReduce</i>	<i>Spark</i>
Hadoop MapReduce stores data on disk	Spark stores data in-memory i.e. first whole data is store into memory and then process it
disk based memory computing i.e. partial use of memory	RAM memory based computing i.e. partial use of disk
It is for batch only	support interactive query
MapReduce uses replication for fault tolerance	Resilient Distributed Datasets (RDD) for fault tolerance
It is used for generating report help to find answer from historical queries	It perform streaming, batch processing and machine learning
It is slower than spark	It is 100 time faster than MapReduce in execution
It is highly painful when it comes to processing and analyzing data in real time	Spark can be used to modify the data in real time
Hadoop handles in low cost	Cost-wise Spark requires a more cost in
MapReduce is faster in batch processing	If the data size is greater than memory it will be far slower in the batch processing
It merge and partition shuffle spill files into one big file	SPARK doesn't merge and partition shuffle files
MapReduce is inefficient for applications that repeatedly reuse a same set of data	Keep working sets in memory for efficient reuse
Writing Hadoop MapReduce code is complex and lengthy	Writing Spark code is always compact

A. Map and Reduce phase in MapReduce

The Map phase of Hadoop MapReduce some steps are

Source of data can process on spark:

- HDFS
- AWS S3
- NOSQL : HBASE, Cassandra, MongoDB
- Streaming Component : Kafka, Flume ,etc
- Kinesis



- RDBMS
- NFS (Network File System)
- LFS / NFS
- Teradata
- Hive



Programming Languages support to Spark -

Scala , Java , Python and R
Sparks provided support for above 4 languages.

What type of data process with spark?

1. Structure
2. Semi-structure
3. Non-structure / Unstructured

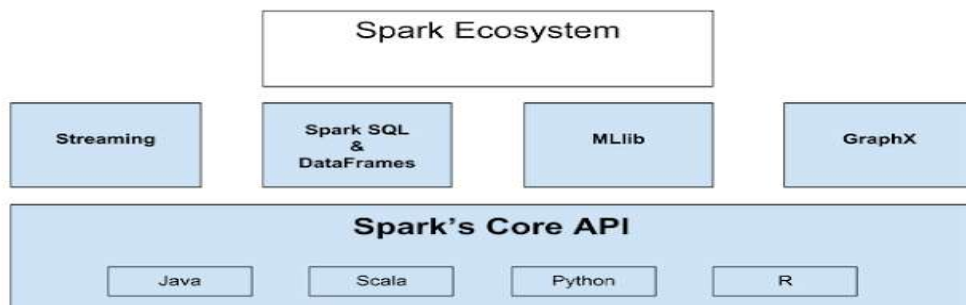
Components of Spark / Ecosystem / Paradigms / Architecture:

To process various type of data there is many component of spark available. such as

1. **Spark Core** - unstructured , Batch Data
2. **Spark SQL** - Structured, Semi-structural , Batch Data



3. **Spark Streaming** - Streaming or real time Data
4. **Spark GraphX** - For Graphical Data, conn. Between nodes etc, Fb, WhatsApp etc.
5. **Spark ML** - Machine Learning Algorithms



Installation :

1. Spark installation on Local – CLI
 2. AWS EMR
 3. Google Colab
 4. Local – Pycharm
 - 5- Databricks
- Etc.

Spark Core

Introduction -

All the functionalities being provided by Apache Spark are built on the top of **Spark Core**.

It delivers speed by providing in-memory computation capability. Thus Spark Core is the foundation of parallel and distributed processing of huge dataset.

Spark core is a “computational engine” that is responsible for scheduling, distributing, and monitoring



applications consisting of many computational tasks across many worker machines, or a computing cluster.

The key features of Apache Spark Core are:

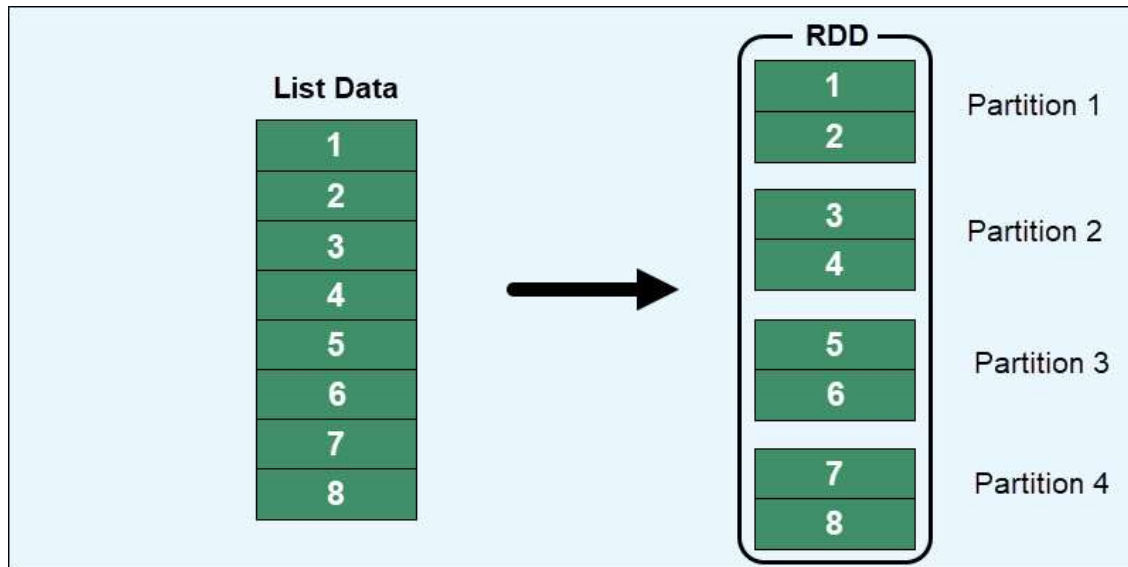
- It is in charge of essential I/O functionalities.
- Significant in programming and observing the role of the **Spark cluster**.
- Task dispatching.
- Fault recovery.
- It overcomes the snag of **MapReduce by using in-memory computation**.
- **Spark Core** is embedded with a special collection called **RDD (resilient distributed dataset)**

Functions of Spark Core:

- Spark Core is the base engine for large-scale parallel and distributed data processing.
- The core is the distributed execution engine and the Java, Scala, and Python APIs offer a platform for distributed ETL application development. SparkCore performs various important functions like
 - memory management,
 - monitoring jobs,
 - fault-tolerance,
 - job scheduling and
 - interaction with storage systems. Further, additional libraries, built a top the core allow diverse workloads for streaming, SQL, and machine learning.

What is RDD :

RDD stands for Resilient Distributed Dataset, and it is a fundamental data structure in Apache Spark. RDD is the core abstraction that represents a distributed collection of data items that can be processed in parallel across a cluster. RDDs provide fault tolerance through lineage information, which allows them to recover lost data due to node failures.



RDD further divided into partitions :

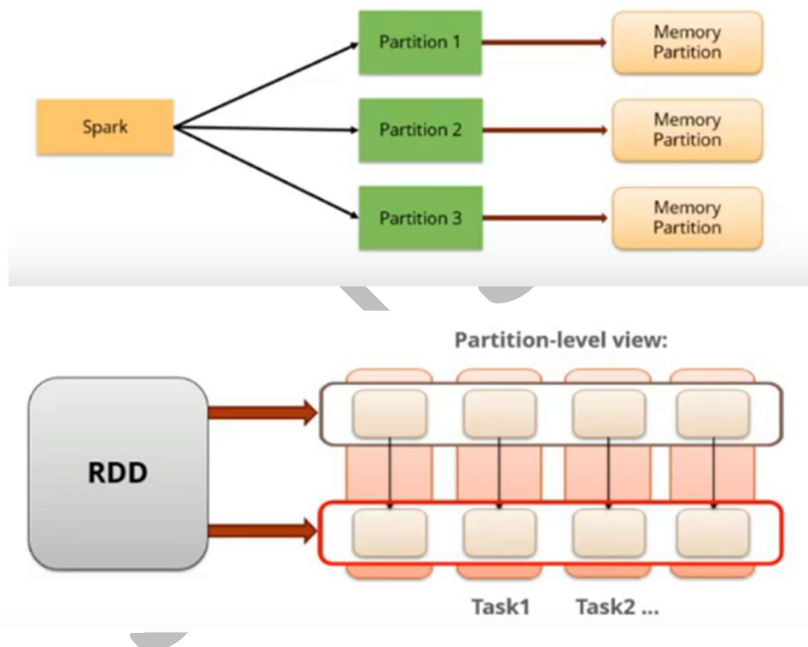
Partitions in RDDs (Resilient Distributed Datasets) are the basic units of parallelism in Apache Spark. An RDD is logically divided into a set of partitions, where each partition is a collection of data that resides on a specific node in a cluster. Partitions are the units of parallelism, and they enable Spark to process different portions of the data in parallel across the nodes of a cluster.

Partitions :

- RDD is divided into partitions
- By using partitions we achieve the parallelism .
- **Spark** automatically **partitions** RDDs and distributes the **partitions** across different nodes
- In short we can say it is logical division of data distributed across all nodes.
- The number of partitions in spark are configurable and having too few or too many partitions is not good.
- If an RDD has too many partitions, then task scheduling may take more time than the actual execution time.
- The lower bound for spark partitions is determined by 2 X number of cores in the cluster available to application.



- the upper bound for partitions in Spark, the task should take 100+ ms time to execute. If it takes less time, then the partitioned data might be too small
- While process data each partitions assign to each worker node. But partitions can not split on multiple nodes
- All Data objects (RDD,Dataframe,Datasets,DStream,Vector,GraphFrames) are divided into partitons
- Number of partitions is depend on
 - Cluster size
 - File System
- If the number of partitions more than dataset then some partitions will get empty.
- By Default spark read data from HDFS
- Each Partitions / Split by default in RAM



Sample Program :

1.

```
ls=[10,20,30,40]
```

```
rdd=spark.sparkContext.parallelize(ls)
```



```
rdd1=rdd.map(lambda x:x*10)
```

```
print(rdd1.collect())
```

2.

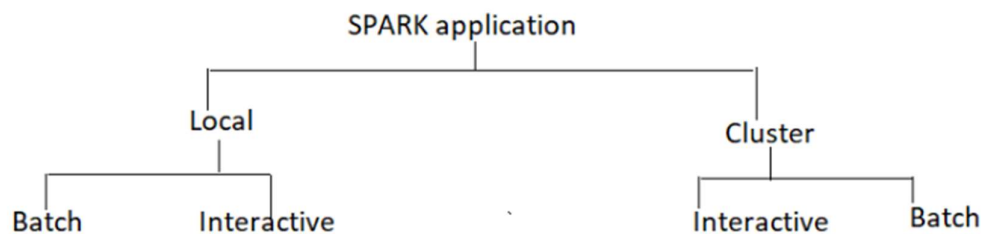
```
rdd=spark.sparkContext.textFile("file:///E:/data.txt")
```

```
rdd1=rdd.map(lambda x:x+" is clever student")
```

```
print(rdd1.collect())
```

```
rdd1.saveAsTextFile("file:///E:/spop")
```

Spark Mode :



Application can be launched in 2 modes

1. Interactive Mode (REPL : read evaluate print loop) (spark-shell)

Launch Spark shell in interactive mode

Cmd prompt or shell (spark-shell)

X=5

Y=6

X+Y=11 (REPL)

2. Batch Mode (spark-submit)

In batch mode we write application , create jar and run it on local or cluster

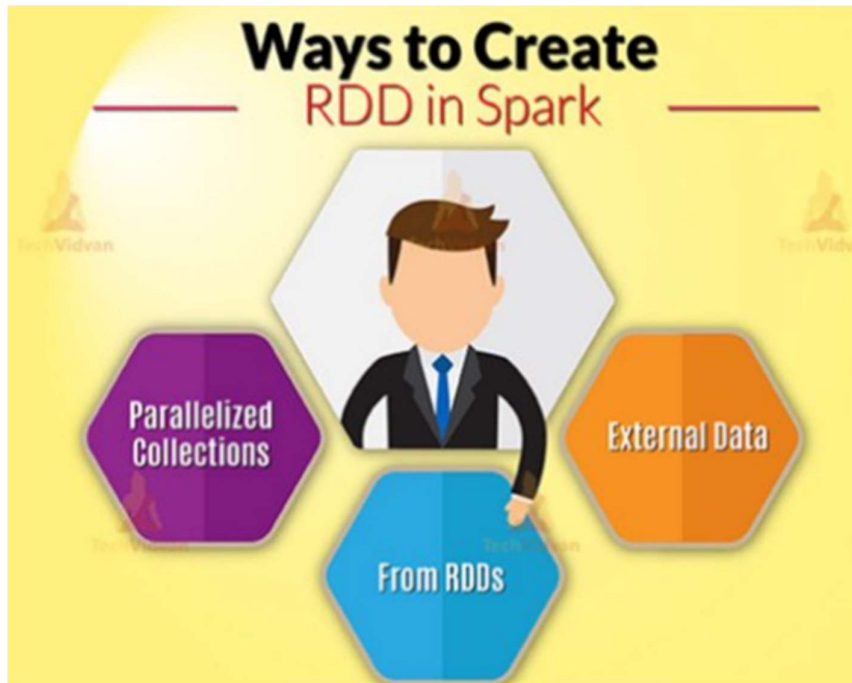
Creation of RDD



There are three ways to create an RDD in Spark.

1. Parallelizing already existing collection in driver program.
2. Referencing a dataset in an external storage system (e.g. HDFS, Hbase, shared file system).
3. Creating RDD from already existing RDDs.

Ways to create RDD :



1.Parallelized collection (parallelizing) or makeRDD

RDDs are generally created by parallelized collection i.e. by taking an existing collection in the program and passing it to SparkContext's parallelize() method.

Exm-

```
t=(100,200,300,400)
```

```
rdd=spark.sparkContext.parallelize(t)
```

```
print(t.collect())
```



2.External Datasets (Referencing a dataset)

In Spark, the distributed dataset can be formed from any data source supported by Hadoop, including the local file system, HDFS, Cassandra, HBase etc. In this, the data is loaded from the external dataset. To create text file RDD, we can use SparkContext's `textFile` method. It takes URL of the file and read it as a collection of line. URL can be a local path on the machine or a `hdfs://`, `s3n://`, etc.

```
rdd=spark.sparkContext.textFile("file:///E:/emp.txt")
rdd1=rdd.filter(lambda x:'pune' in x)
rdd1.collect()
rdd1.saveAsTextFile("file:///E:/emp_pune")
```

3. Creating RDD from existing RDD

Transformation mutates one RDD into another RDD, thus transformation is the way to create an RDD from already existing RDD.

Transformation acts as a function that intakes an RDD and produces one. The input RDD does not get changed, because RDDs are immutable in nature but it produces one or more RDD by applying operations. Some of the operations applied on RDD are: filter, count, distinct, **Map, FlatMap** etc.

```
t=(100,200,300,400)
rdd=spark.sparkContext.parallelize(t)
rdd1=rdd.map(lambda x:x*10)
print(rdd1.collect())
```

Operations Perform on RDD

There is 2 types of operation we can perform on RDD or any data objects (even other too DF, DS, Dstream, Vector, Graph Frames)

A. Transformation :

- When we create one RDD from another is nothing but Transformation.
- Transformation(RDD)=RDD



- Transformation are always **Lazy** in nature means when we perform any action on RDD that time it execute transformation.
- RDD is immutable so each time it creates a new RDD

There are two types of transformations:

1. Narrow transformation

2. Wide transformation

1. Narrow transformation

In Narrow transformation, all the elements that are required to compute the records in single partition live in the single partition of parent RDD. A limited subset of partition is used to calculate the result. Narrow transformations are the result of `map()`, `filter()`.

In Narrow Transformation Shuffling between the partitions is not occurs.

It is light weighted transformation

2. Wide transformation

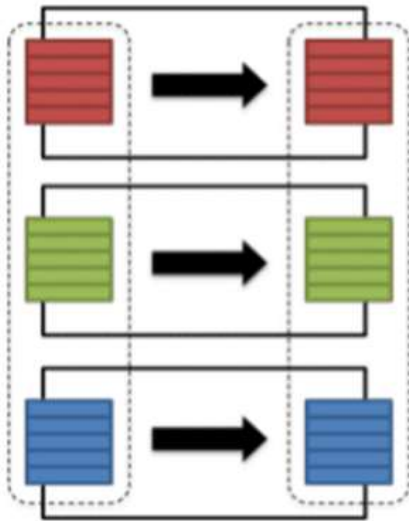
In wide transformation, all the elements that are required to compute the records in the single partition may live in many partitions of parent RDD. The partition may live in many partitions of parent RDD. Wide transformations are the result of `groupByKey()` and `reduceByKey()`.

It is Highly weighted transformation

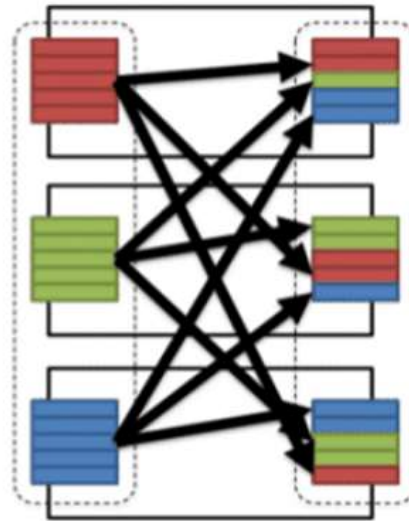
Two or more partitions exchange the Data.



Narrow



Wide

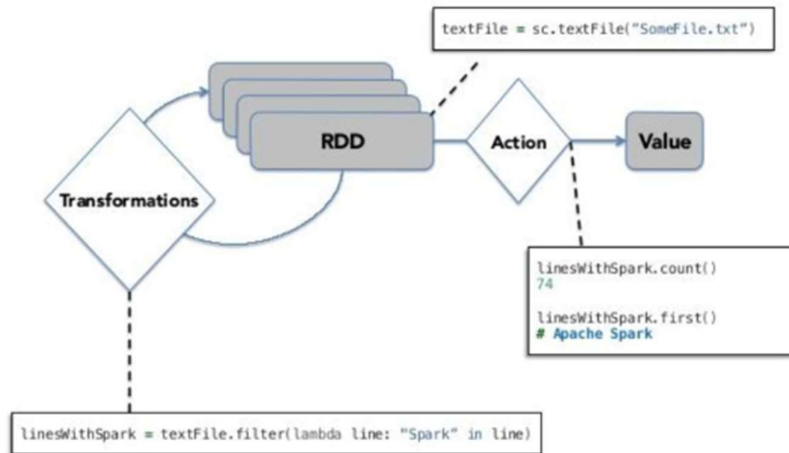


B. Actions :

- When we perform action on RDD it gives Value or collections.
- Action(RDD) = Value or Collection
- It evaluate instantly



Working With RDDs



Exm –

#reading file

```
rdd=spark.sparkContext.textFile("file:///E:/inputdata.txt")
```

```
rdd1=rdd.flatMap(lambda x:x.split(" "))
```

```
rdd2=rdd1.map(lambda x:(x,1))
```

```
rdd3=rdd2.reduceByKey(lambda x,y:x+y)
```

```
rdd3.collect()
```

OR

```
rdd=spark.sparkContext.textFile("file:///E:/inputdata.txt")
```



```
rdd1=rdd.flatMap(lambda x:x.split(" ")).map(lambda x:(x,1)).reduceByKey(lambda x,y:x+y)
```

```
rdd1.collect()
```

Paired RDD / Key value pair RDD :

Spark Paired RDDs are nothing but RDDs containing a key-value pair. Basically, key-value pair (KVP) consists of a two linked data item in it. Here, the key is the identifier, whereas value is the data corresponding to the key value.

Moreover, Spark operations work on RDDs containing any type of objects. However key-value pair RDDs attains few special operations in it. Such as, distributed “shuffle” operations, grouping or aggregating the elements by a key.

In addition, on Spark Paired RDDs containing Tuple2 objects in Scala, these operations are automatically available. Basically, operations for the key-value pair are available in the Pair RDD functions class. However, that wraps around a Spark RDD of tuples.

How to Create Spark Paired RDD

There are several ways to create Paired RDD in Spark, like by running a map() function that returns key-value pairs. However, language differs the procedure to build the key-value RDD. Such as

Exm –
input
spark
is
easy

```
rdd2=rdd1.map(lambda x:(x,1))
```

spark,1
is ,1
easy,1

how we can create :

```
rdd2=rdd1.map(lambda x:(x,1))
```



[(spark,1),(is,1),(spark,1),(is,1)]

why we create paired RDD:

1.on paired RDD we can perform special operation

ex. ReduceByKey()

Exp :

#email

john.doe@gmail.com,sarah.smith@yahoo.com,alex.jones@hotmail.com,emma.wilson@gmail.com,samuel.roberts@gmail.com,lisa.miller@yahoo.com,chris.evans@hotmail.com,amy.jackson@gmail.com,jason.cook@yahoo.com,olivia.wright@hotmail.com,kevin.adams@gmail.com,sophie.brown@yahoo.com,morgan.white@hotmail.com,daniel.green@gmail.com,laura.morris@yahoo.com,brian.hall@hotmail.com,natalie.clark@gmail.com,peter.ward@yahoo.com,mia.anderson@hotmail.com,jacob.fisher@gmail.com,ava.hill@yahoo.com,ethan.taylor@hotmail.com,grace.cooper@gmail.com,matthew.davis@yahoo.com,emily.jones@hotmail.com

Code :

```
rdd=spark.sparkContext.textFile("file:///E:/emails.txt")
```

```
rdd1=rdd.flatMap(lambda x:x.split(","))
```

```
rdd2=rdd1.map(lambda x:x.split("@")[1])
```

```
rdd22=rdd2.map(lambda x:x.upper())
```

```
rdd3=rdd22.map(lambda x:(x,1))
```

```
rdd4=rdd3.reduceByKey(lambda x,y:x+y)
```

```
rdd4.collect()
```

#Transformation List



map

flatMap

reduceByKey

filter

intersection

union

subtract

join

r1.join(r2).collect()

#action List

collect

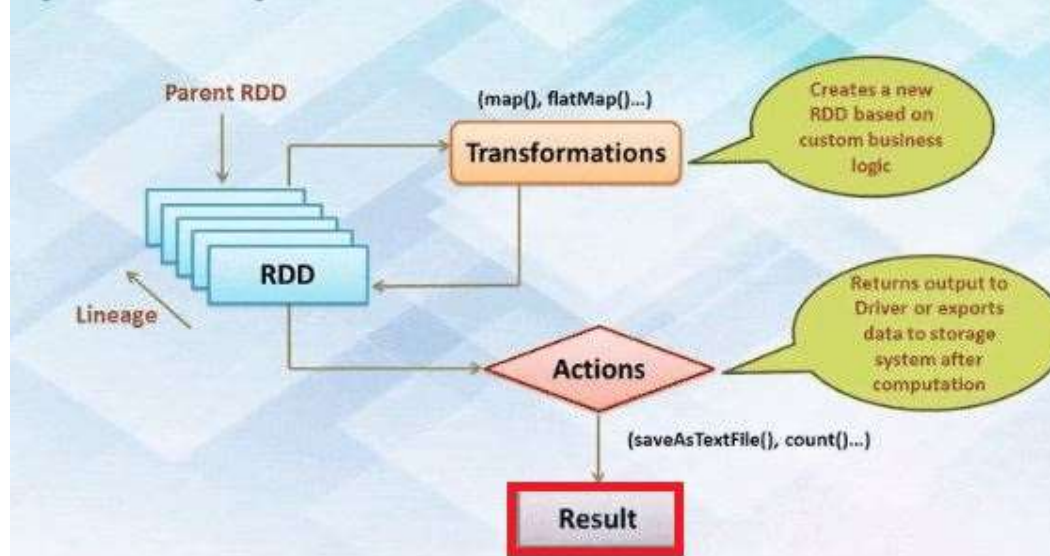
count

take(n)

Lazy Evaluation :



Spark Lazy Evaluation



As the name itself indicates its definition, lazy evaluation in Spark means that the execution will not start until an action is triggered.

In Spark, the picture of lazy evaluation comes when Spark transformations occur. Transformations are lazy in nature meaning when we call some operation in RDD, it does not execute immediately. Spark maintains the record of which operation is being called (Through DAG). We can think Spark RDD as the data, that we built up through transformation. Since transformations are lazy in nature, so we can execute operation any time by calling an action on data. Hence, in lazy evaluation data is not loaded until it is necessary.

Advantages of Lazy Evaluation in Spark Transformation

- Increase Manageability: The Spark Lazy evaluation, users can divide into smaller operations. It reduces the number of passes on data by transformation grouping operation.
- Increases Speed: By lazy evaluation in Spark to saves the trip between driver and cluster, speed up the process.
- Reduces Complexities: There are two types of complexities of any operations are Time and Space complexity using Spark lazy evaluation we can overcome both complexities. The action is triggered only when the data is required.
- optimization



Spark Installation on Pycharm –

Repartition :

In PySpark, repartitioning is the process of redistributing the data across the partitions of a DataFrame or Resilient Distributed Dataset (RDD). Repartitioning can be useful for optimizing the performance of certain operations, such as join operations, by ensuring that related data is colocated in the same partition.

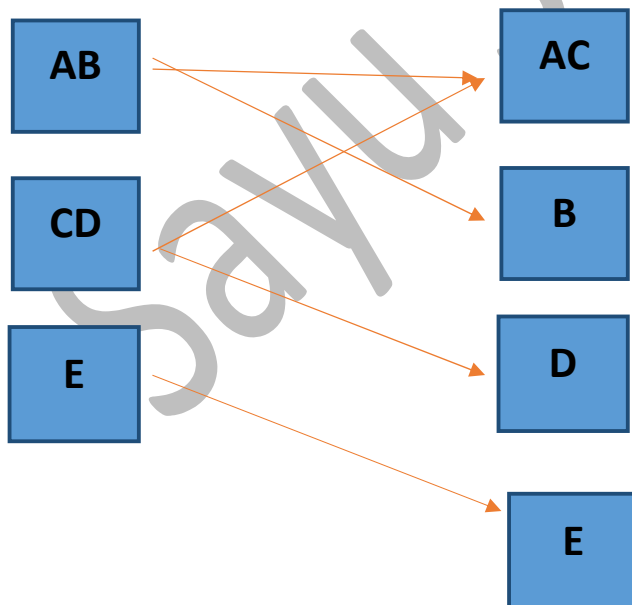
Useful to increase or decrease of partitions .

Syntax :

`Rdd.repartition(n)`

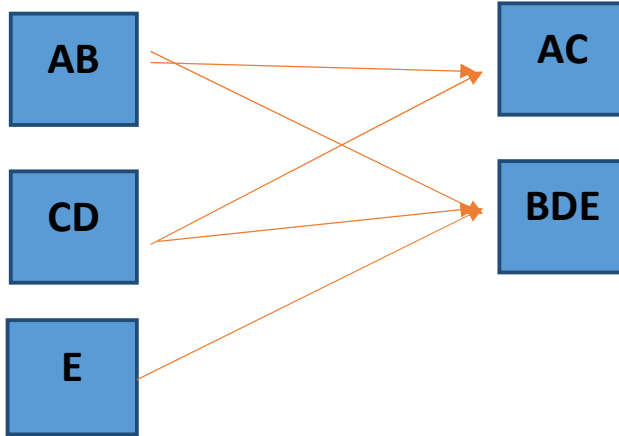
n : number of partitions which you wanted to change.

Increase no of partition using repartition : `rdd1=rdd.repartition(4)`





Decrease no of partition using repartition : `rdd1=rdd.repartition(2)`



Coalesce :

In PySpark, the **coalesce** method is used to reduce the number of partitions in a DataFrame without shuffling the data across the cluster. This is particularly useful when you want to decrease the number of partitions without incurring the cost of a full shuffle, which can be expensive.

The **coalesce** method is less resource-intensive than **repartition** when reducing the number of partitions because it avoids the need to move data between nodes. It is a more efficient option when the goal is to decrease the number of partitions.

Syntax :

```
Rdd1=rdd.coalesce(2)
```

2 : resultant rdd no of partitions





Repartition	Coalesce
• Can Both Increase and Decrease number of partitions • Does not Worry about amount of shuffle • Slow than Coalesce • Tries to create partitions of similar size	• Used To decrease number of partitions • Tries to avoid shuffle • Better Performance than Repartition • Output Partition can be uneven in size

Persist and cache() :

Spark RDD persistence is an optimization technique in which saves the result of RDD evaluation.

Using this we save the intermediate result so that we can use it further if required.

It reduces the computation overhead

We can make persisted RDD through cache() and persist() methods.

When we use the cache() method we can store all the RDD in-memory.

We can persist the RDD in memory and use it efficiently across parallel operations.

The difference between cache() and persist() is that using cache() the default storage level is MEMORY_ONLY while using persist() we can use various storage levels

The cache memory of the Spark is fault tolerant so whenever any partition of RDD is lost, it can be recovered by transformation Operation that originally created it.

If the persisted data is lost that time spark have to regenerate from the start.



If we persist data in RAM that time its faster than the Disk. reading and writing is faster

Persistence info. Store in Storage tab of application master.

When we go with persistence

In 2 Scenario

1. Transaformation is complex (req. More time to create)
2. If we use RDD in multiple flows.

Benefits of RDD Persistence in Spark

Time efficient

Cost efficient

Lesser the execution time.

How can we use persist() and cache():

```
var r1=sc.textFile("hdfs://file.txt")
```

```
R1.cache()
```

```
r1.persist()
```

```
Rdd2.persist(StorageLevel.MEMORY_ONLY)
```

```
Rdd2.persist(StorageLevel.MEMORY_ONLY_SER)
```

```
Rdd2.persist(StorageLevel.MEMORY_AND_DISK)
```

```
Rdd2.persist(StorageLevel.MEMORY_AND_DISK_SER)
```

```
Rdd2.persist(StorageLevel.DISK_ONLY)
```

```
Rdd2.persist(StorageLevel.DISK_ONLY_2)
```

```
Rdd2.persist(StorageLevel.MEMORY_ONLY_2)
```

In above 2 is nothing but data persist in 2 copies. Maximum support the two copies only till now.

Green color spot in DAG graph is nothing but the particular RDD is persisted.

Storage levels of Persisted RDDs :

a. MEMORY_ONLY

In this storage level, RDD is stored as deserialized Java object in the JVM. If the size of RDD is greater than memory, It will not cache some partition and recompute them next time whenever needed. In this level the space used for storage is very high, the CPU computation time is low, the data is stored in-memory. It does not make use of the disk.

BY default the persist() method store into the memory



Ex. Rdd2.persist()

Or

Rdd2.persist(StorageLevel.MEMORY_ONLY)

RDD persist only in memory.

To unpersist the RDD we can use below method.

Rdd2.unpersist()

b. MEMORY_ONLY_SER

This level of Spark store the RDD as serialized Java object (one-byte array per partition). It is more space efficient as compared to deserialized objects, especially when it uses fast serializer. But it increases the overhead on CPU. In this level the storage space is low, the CPU computation time is high and the data is stored in-memory. It does not make use of the disk.

Rdd2.persist(StorageLevel.MEMORY_ONLY_SER)

Rdd2.unpersist()

When Data is want to process again so it should deserilized again so it require CPU.

c.MEMORY_AND_DISK

It does not mean the RDD is stored in both memory and disk. First preference is the Memory if space is not sufficient to store RDD in memory that time exceeded data it stored into the Disk.

In this level, RDD is stored as deserialized Java object in the JVM. When the size of RDD is greater than the size of memory, it stores the excess partition on the disk, and retrieve from disk whenever required. In this level the space used for storage is high, the CPU computation time is medium, it makes use of both in-memory and on disk storage

d.MEMORY_AND_DISK_SER

It is similar to **MEMORY_ONLY_SER**, but it drops the partition that does not fits into memory to disk, rather than recomputing each time it is needed. In this storage level, The space used for storage is low, the CPU computation time is high, it makes use of both in-memory and on disk storage.

Rdd2.persist(StorageLevel.MEMORY_AND_DISK_SER)

Rdd2.unpersist()

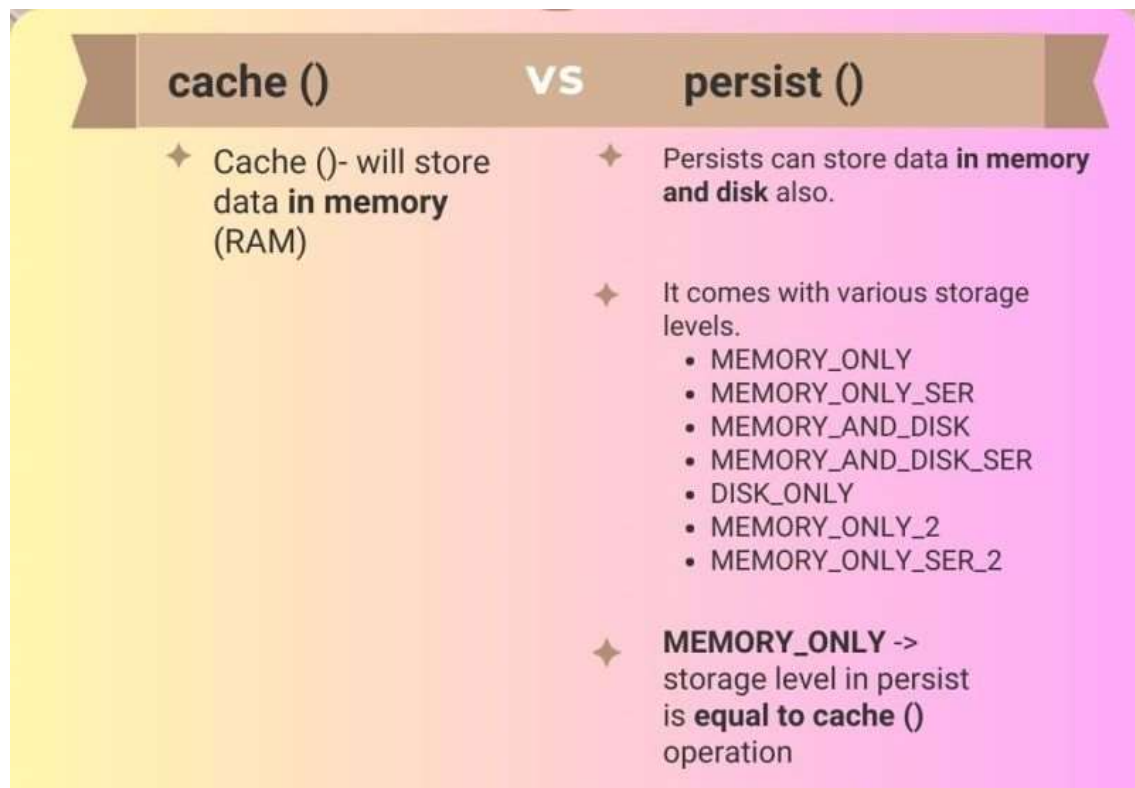


e.DISK_ONLY

In this storage level, RDD is stored only on disk. The space used for storage is low, the CPU computation time is high and it makes use of on disk storage.

`Rdd2.persist(StorageLevel.DISK_ONLY)`

`Rdd2.unpersist()`



DAG :

DAG is nothing but Directed acyclic graph is a graph which save the information about sequence of operations performed on the Target RDD.

When we applied transformation nothing happen

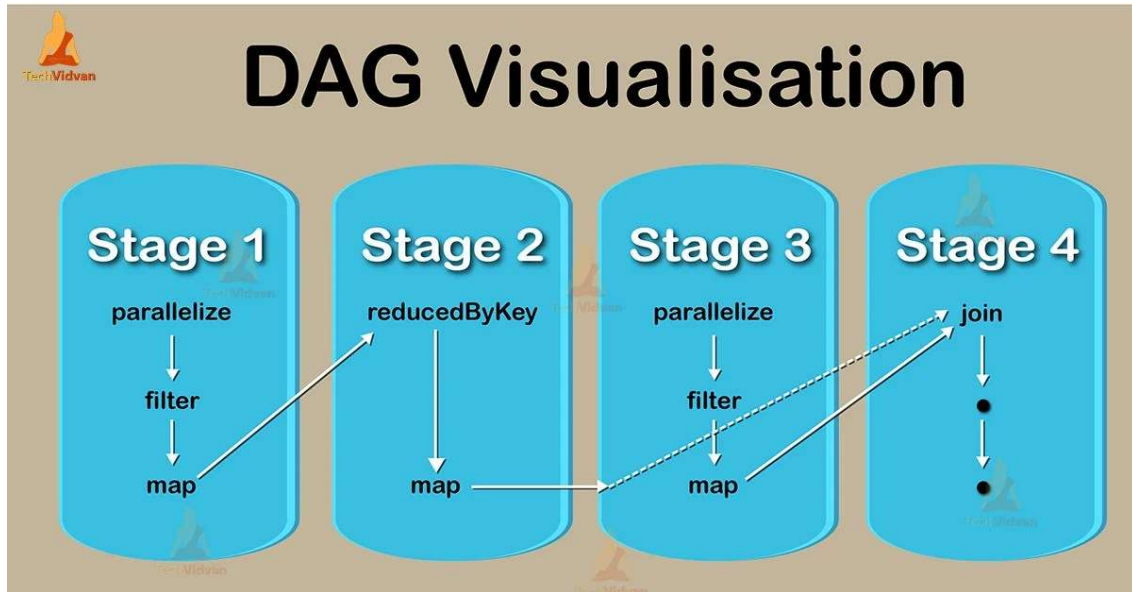
but when we applied action on RDD that time DAG is created.

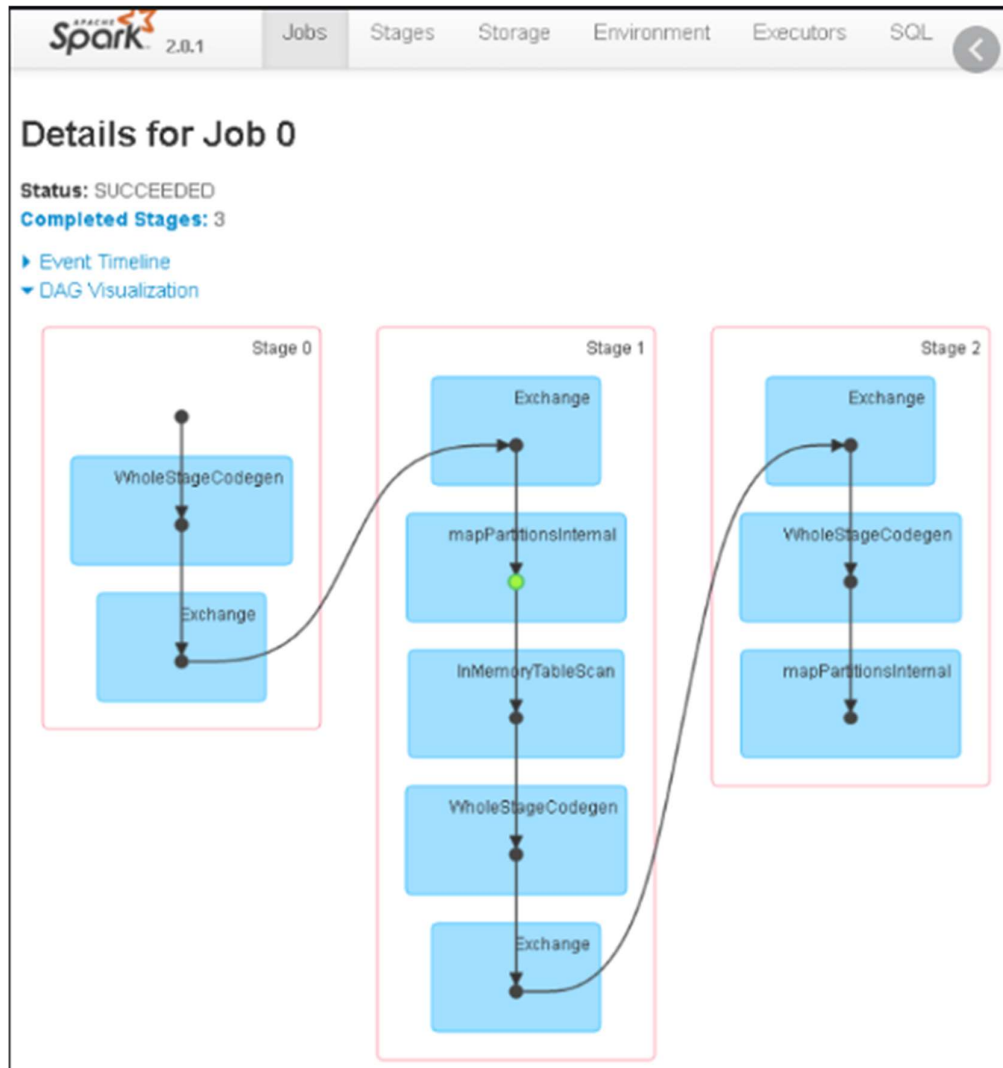
After called action the Created Graph is submitted to the DAG scheduler which further split into Graph into stages and the Tasks



(Directed Acyclic Graph) DAG in Apache Spark is a set of Vertices and Edges, where vertices represent the RDDs and the edges represent the Operation to be applied on RDD.

In Spark DAG, every edge directs from earlier to later in the sequence.





We can see DAG at browser 8088 or 4040 in job --> DAG tab

How DAG works in Spark?

- The interpreter is the first layer, using a Scala interpreter, Spark interprets the code with some modifications.
- Spark creates an operator graph when you enter your code in Spark console.
- When we call an **Action** on Spark RDD at a high level, Spark submits the operator graph to the **DAG Scheduler**.
- Divide the operators into **stages** of the task in the DAG Scheduler. A stage contains task based on the partition of the input data. The DAG scheduler



pipelines operators together. For example, map operators schedule in a single stage.

- The stages pass on to the **Task Scheduler**. It launches task through **cluster manager**. **The dependencies of stages are unknown to the task scheduler.**
- The **Workers** execute the task on the slave.

At higher level, we can apply two type of RDD transformations: **narrow transformation** (e.g. map(), filter() etc.) and **wide transformation** (e.g. reduceByKey()). *Narrow transformation* does not require the shuffling of data across a partition, the narrow transformations will group into single stage while in *wide transformation* the data shuffles. Hence, Wide transformation results in stage boundaries.

Each RDD maintains a pointer to one or more parent along with metadata about what type of relationship it has with the parent. For example, if we call `val b=a.map()` on an RDD, the RDD *b* keeps a reference to its parent RDD *a*, that's an **RDD lineage**.

How to Achieve Fault Tolerance through DAG

RDD splits into the partition and each node operates on a partition at any point in time. Here, the series of **Scala function executes on a partition of the RDD**.

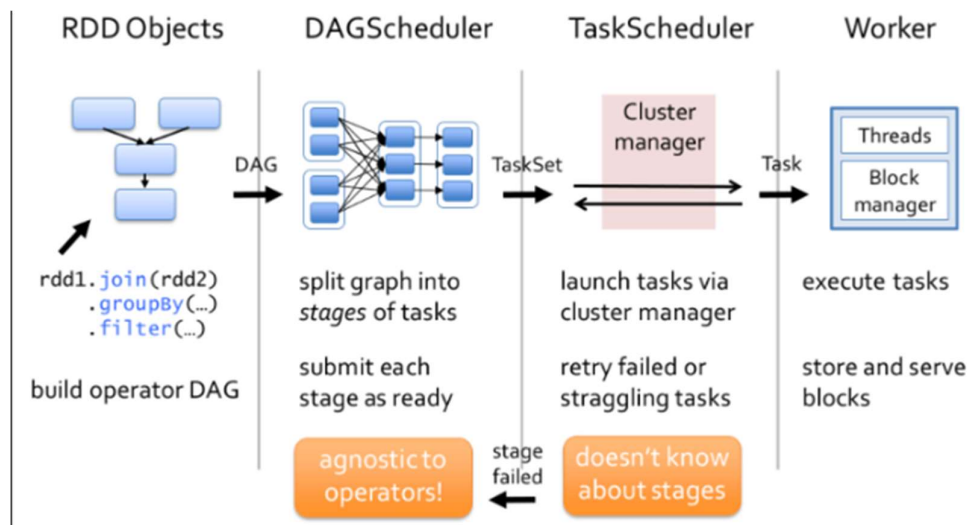
These operations compose together and Spark execution engine view these as DAG (Directed Acyclic Graph).

When any node crashes in the middle of any operation say O3 which depends on operation O2, which in turn O1. The *cluster manager* finds out the node is dead and assign another node to continue processing. This node will operate on the particular partition of the RDD and the series of operation that it has to execute (O1->O2->O3). Now there will be no data loss.

Advantages of DAG in Spark

There are multiple advantages of Spark DAG, let's discuss them one by one:

- The lost RDD can recover using the Directed Acyclic Graph.
- Map Reduce has just two queries the map, and reduce but in DAG we have multiple levels. So to execute SQL query, DAG is more flexible.
- DAG helps to achieve fault tolerance. Thus we can recover the lost data.
- It can do a better global optimization than a system like Hadoop MapReduce.



DAG Scheduler

Working of DAG Scheduler

It is a scheduling layer in a **spark** which implements stage oriented scheduling. It converts logical execution plan to a physical execution plan. When an action is called, **spark** directly strikes to **DAG scheduler**. It executes the tasks those are submitted to the **scheduler**

It creates the Physical plan

It divided whole execution in stages

Task Scheduler :

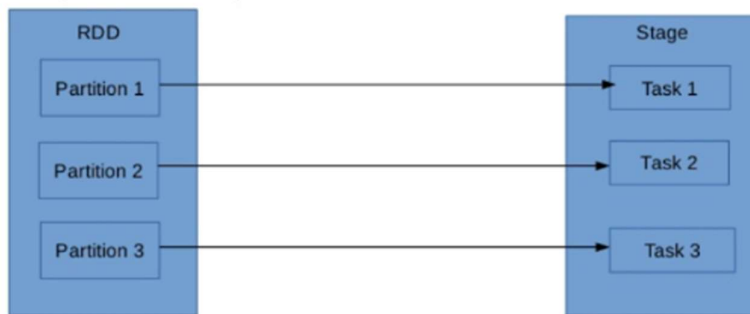
As a core component of data processing platform, **scheduler** is responsible for schedule **tasks** on compute units. Built on a Directed Acyclic Graph (DAG) compute model, **Spark Scheduler** works together with Block Manager and Cluster Backend to efficiently utilize cluster resources for high performance of various workloads.

Low-level task scheduler interface, currently implemented exclusively by TaskSchedulerImpl. This interface allows plugging in different task schedulers. Each TaskScheduler schedules tasks for a single SparkContext. These schedulers get sets of tasks submitted to them from the DAGScheduler for each stage, and are responsible for sending the tasks to the cluster, running them, retrying if there are failures, and mitigating stragglers. They return events to the DAGScheduler.



Tasks, Stages, Jobs

- **Tasks:** smallest individual unit of execution that represents a partition in a dataset.



Lineage Graph

it stores only information about flow of the Data means dependency between the RDD.

It not store the actual Data.

Spark lineage is the set of steps (Transformations) to require to regenerate RDD.

Lineage is the logical plan which applies on RDD to create a final RDD.

DAG is a physical plan , and DAG has more information like how diff stage is dependent on each other, which stages can run parallel, how stages will execute etc.

```
rdd1
|
rdd2
|
rdd3
```

RDD lineage is nothing but the graph of all the parent RDDs of an RDD. We also call it an RDD operator graph or RDD dependency graph. To be very specific, it is an output of applying transformations to the spark. Then, it creates a logical execution plan.

Also, physical execution plan or execution DAG is known as DAG of stages.

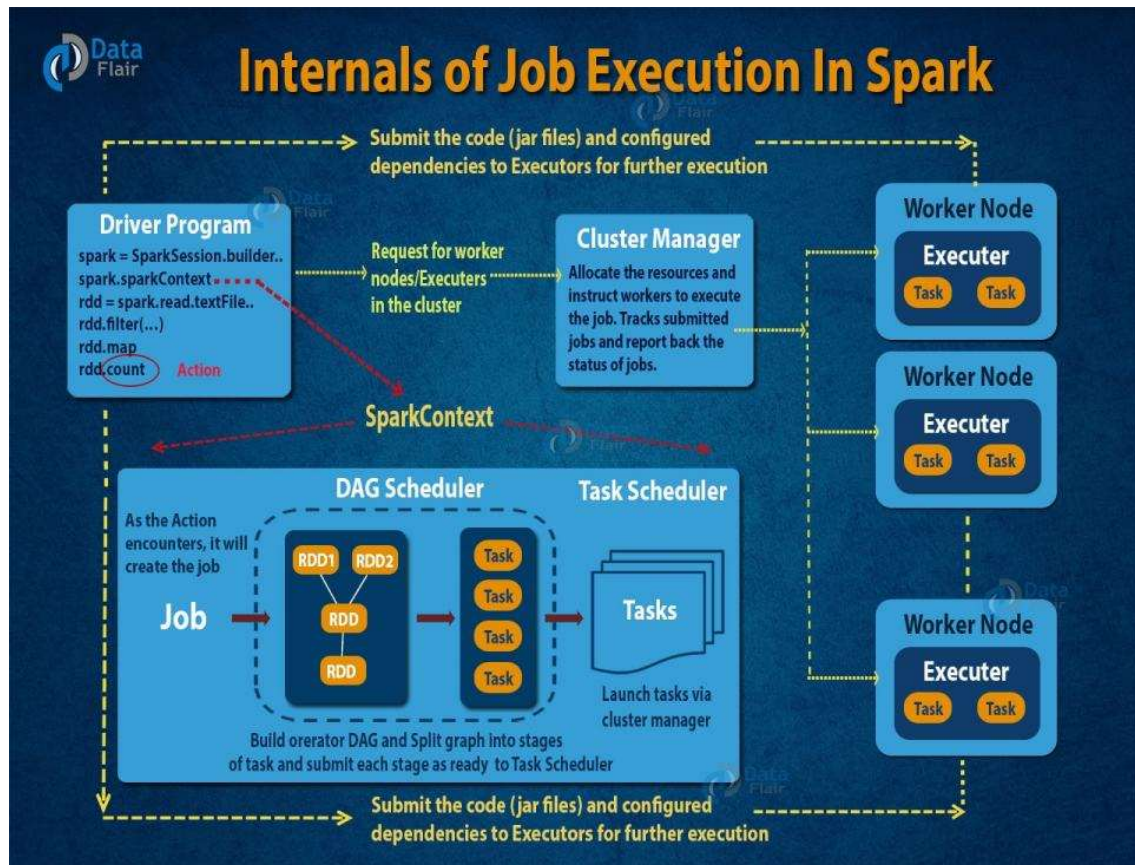
ToDebugString Method to get RDD Lineage Graph in Spark

toDebugString: String

exm :



rdd3.toDebugString



Email program –

Input –

john.doe@gmail.com,sarah.smith@yahoo.com,alex.jones@hotmail.com,emma.wilson@gmail.com,samuel.roberts@gmail.com,lisa.miller@yahoo.com,chris.evans@hotmail.com,amy.jackson@gmail.com,jason.cook@yahoo.com,olivia.wright@hotmail.com,kevin.adams@gmail.com,sophie.brown@yahoo.com,morgan.white@hotmail.com,daniel.green@gmail.com,laura.morris@yahoo.com,brian.hall@hotmail.com,natalie.clarke@gmail.com



rk@gmail.com,peter.ward@yahoo.com,mia.anderson@hotmail.com,jacob.fisher@gmail.com,ava.hill@yahoo.com,ethan.taylor@hotmail.com,grace.cooper@gmail.com,matthew.davis@yahoo.com,emily.jones@hotmail.com

Code :

```
from pyspark.sql import SparkSession
spark=SparkSession.builder.appName("Spark Testing").getOrCreate()
rdd=spark.sparkContext.textFile("file:///E:/emails.txt")
rdd1=rdd.flatMap(lambda x:x.split(","))
rdd2=rdd1.map(lambda x:x.split("@")[1])
rdd3=rdd2.map(lambda x:(x,1))
rdd4=rdd3.reduceByKey(lambda x,y:x+y)
rdd4.collect()
```

Spark Terminologies –

Core –

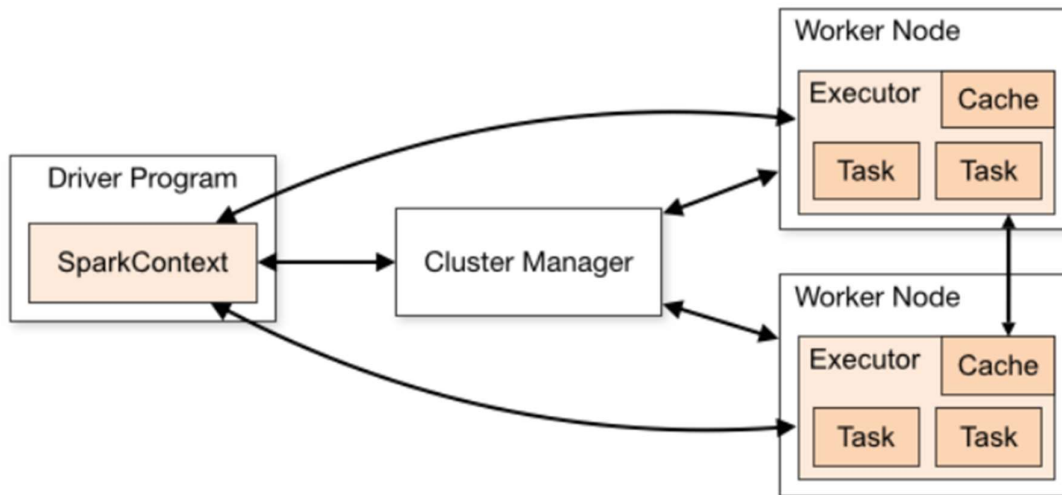
In the context of Apache Spark, "core" typically refers to a computing resource or a processing unit. In Spark, a core is a basic computational unit that represents a single task that can be executed in parallel. It is often associated with a CPU core, which is the basic processing unit of a central processing unit (CPU).

Apache Spark is designed to distribute data processing tasks across a cluster of machines, and each machine in the cluster contributes its processing power, measured in terms of cores. Therefore, when you see references to the number of cores in the context of Spark, it usually means the total number of CPU cores available across all the machines in the Spark cluster.



For example, if you have a cluster with three machines, each having 4 CPU cores, the total number of cores in your Spark cluster would be 3 machines * 4 cores/machine = 12 cores.

Driver :



Driver is a Java process. This is the process where the main() method of our Scala, Java, Python program runs. It executes the user code and creates a SparkSession or SparkContext and the SparkSession is responsible to create DataFrame, DataSet, RDD, execute SQL, perform Transformation & Action, etc.

Responsibility of DRIVER

1. The main() method of our program runs in the Driver process. It creates SparkSession or SparkContext.
2. Conversion of the user code into Task (transformation and action). It looks at the user code and determines the possible Tasks, i.e. the number of tasks to be performed is decided by the Driver. But how does it determine the Tasks? – With the help of Lineage. (Discussed later in the blog)
3. Helps to create the Lineage, Logical Plan and Physical Plan.
4. Once the Physical Plan is generated, the Driver schedules the execution of the tasks by coordinating with the Cluster Manager.



5. Coordinates with all the Executors for the execution of Tasks. It looks at the current set of Executors and schedules our tasks.
6. Keeps track of the data (in the form of metadata) which was cached (persisted) in Executor's (worker's) memory.

EXECUTOR:

Executor resides in the Worker node. Executors are launched at the start of a Spark Application in coordination with the Cluster Manager.

They are dynamically launched and removed by the Driver as per required.

Responsibility of EXECUTOR

To run an individual Task and return the result to the Driver.

It can cache (persist) the data in the Worker node.

Application:

An application in Spark refers to the entire user program that consists of one or more Spark jobs.

It typically starts with creating a SparkContext, defining transformations and actions on RDDs (Resilient Distributed Datasets), and ends with calling an action to trigger the computation.

An application runs on a cluster and coordinates the execution of Spark jobs.

Job:

A job is a high-level computation triggered by an action in a Spark application.

It is a set of tasks that are launched in response to an action (e.g., `collect()`, `count()`) on an RDD.

Jobs are divided based on stages, where each stage represents a set of tasks that can be executed in parallel.



For example, if you perform a `count()` action on an RDD, it might trigger a job with a single stage.

Stage:

A stage is a logical division of a job that represents a set of tasks that can be executed in parallel without any dependencies on each other.

Stages are formed based on narrow transformations (e.g., `map`, `filter`) that can be executed in parallel without shuffling data between partitions.

Stages are separated by wide transformations (e.g., `groupByKey`, `reduceByKey`) that require data shuffling and the movement of data across partitions.

Task:

A task is the smallest unit of computation in Spark and represents the execution of a single operation on a partition of data.

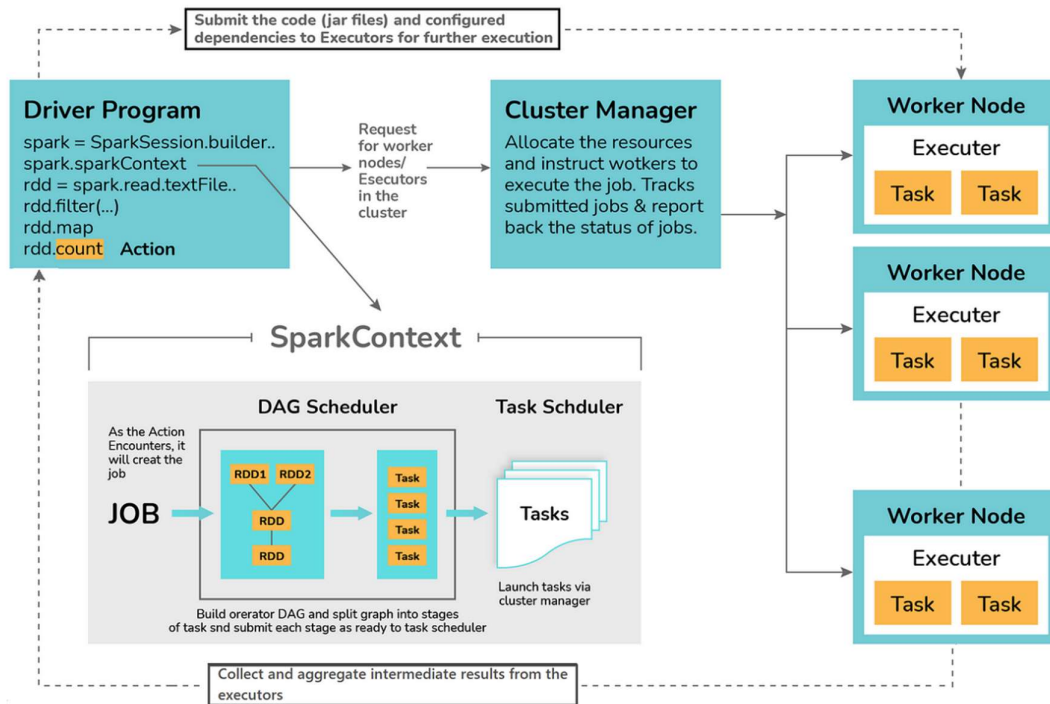
Tasks are the actual units of work that are distributed across the worker nodes in a Spark cluster.

For example, if you have an RDD with 100 partitions and perform a `map` operation on it, there will be 100 map tasks, each processing a separate partition of the data.

Spark Internal / Spark Job Flow :



Internals of Job Execution In Spark



Spark deploy modes

When for execution, we submit a spark job to local or on a cluster, the behaviour of spark job totally depends on one parameter, that is the "Driver" component. Where the "Driver" component of the spark job will reside, it defines the behaviour of the spark job.

There are 2 deployment modes in Apache Spark:-

1. Client mode
2. Cluster mode

1. Client mode



In the mode, the Spark driver component runs on the machine node where we submit the spark job. This mode supports both interactive shells and the job submission commands.

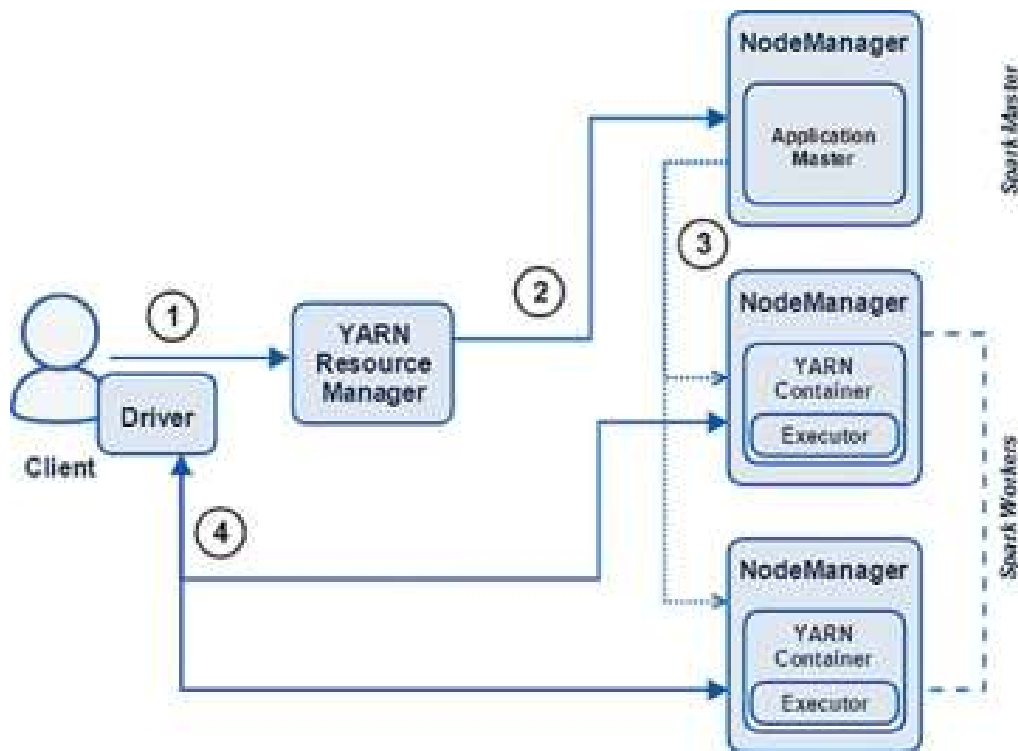
The main disadvantage of the client mode is:-

- If the machine node fails, then the entire job fails.
- The performance of this mode is worst and is not preferred in production environments.

To overcome the worst situations that occur in this mode we use Cluster Mode for deployment in a production environment.

Syntax -

```
spark-submit --deploy-mode client --driver-memory xxxx .....
```





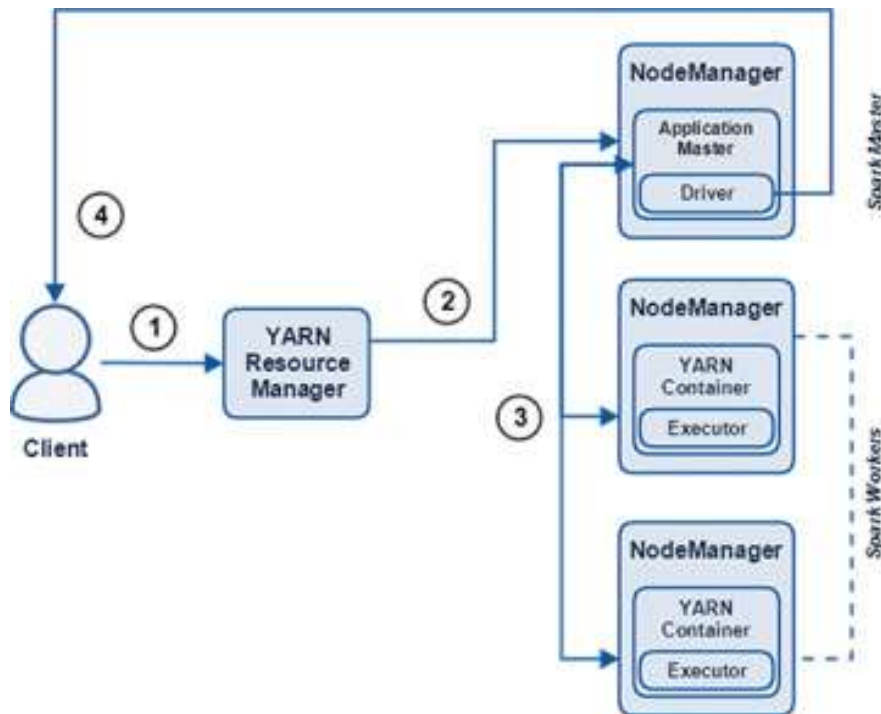
2. Cluster mode

The deploy mode is said to be in cluster mode if the spark job driver component does not run on the machine from which the spark job has been submitted.

- The spark job launches the driver component within the cluster as a part of the sub-process of ApplicationMaster.
- In this mode, we use the spark-submit command for deployment and it does not support interactive shell mode.
- In case the program fails, the driver program is re-instantiated because the driver programs are run in ApplicationMaster.
- In this mode, there is a dedicated cluster manager (such as stand-alone, YARN, Apache Mesos, Kubernetes, etc) for allocating the resources required for the job to run as shown in the below architecture.

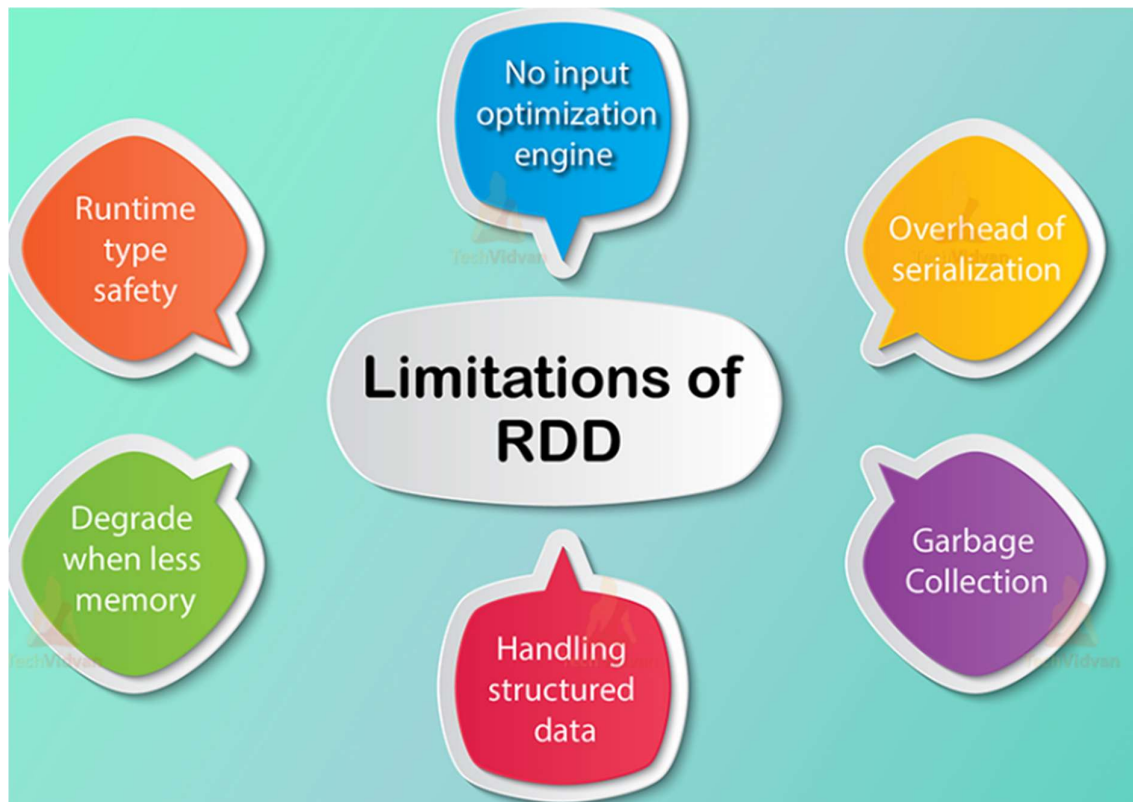
Syntax –

```
spark-submit --deploy-mode cluster --driver-memory xxxx .....
```





Disadvantages of RDD :





1.No Input Optimization Engine

There are several spark advance optimizers like catalyst optimizer and tungsten execution engine. RDDs are not able to use these optimizers. There is no way in RDD for automatic optimization. To optimize, we can optimize each RDD manually.

2. Not Enough Memory

This is a sort of storage issue when we are unable to store RDD due to its lack of memory. The RDD degrades itself when there is not enough space to store spark RDD in-memory or on disk.

All the partitions that are already overflowing from RAM can be later on stored in the disk. Even so, that will provide the same level of performance. To overcome this issue, we need to increase the size of RAM or Disk.

3. Runtime type safety

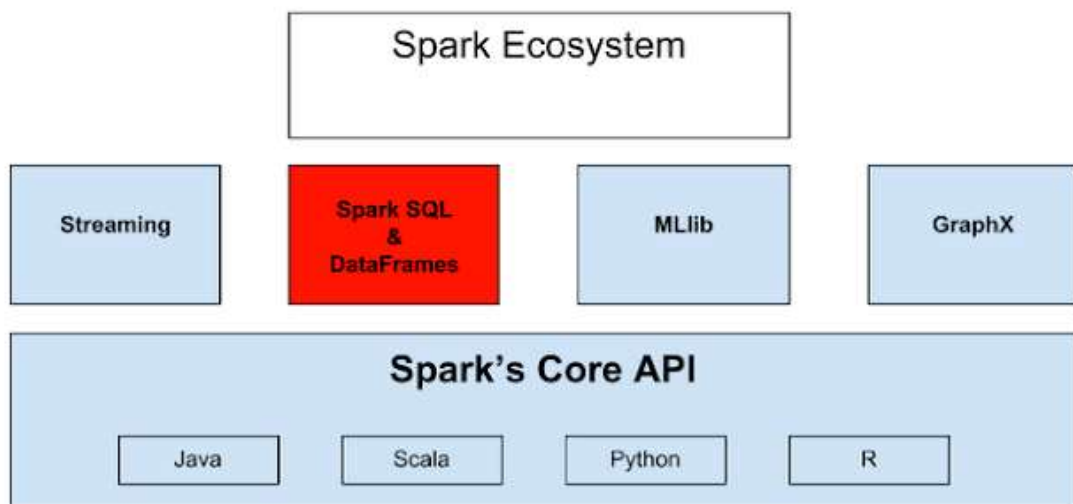
There is no run-time type safety in spark RDD spark RDD. Runtime type safety means it does not allow us to check error at the same time you are compiling the Program.

4. Handling Structured Data

Spark RDD are user specific, they don't infer any schema of the ingested data like other APIs. Such as data frames as well as datasets. Moreover, when we need data immediately, a method to get and import data is data ingestion process.



Spark - SQL



Spark SQL is a module in Apache Spark that provides a programming interface for structured and semi-structured data. It allows developers to query data using SQL-like syntax, enabling seamless integration of SQL queries with Spark programs. Spark SQL supports reading data from various structured sources such as Hive,



Avro, Parquet, ORC, JSON, and relational databases, and it also provides a DataFrame API for working with structured data.

Dataframe –

DataFrame is a distributed collection of data organized into named columns. It is conceptually equivalent to a table in a relational database or a data frame in R/Python, but with richer optimizations under the hood. DataFrames can be constructed from a wide array of sources such as structured data files, tables in Hive, external databases, or existing RDDs

Advantages of Spark SQL –

1. Hive Compatibility -

Spark SQL is compatible with Apache Hive, which means it can query existing Hive tables, enabling organizations to leverage their existing Hive infrastructure and investments while benefiting from the performance improvements and capabilities of Spark.

Performance Optimization –

Spark SQL optimizes queries using Catalyst, an extensible query optimizer. This optimizer leverages the Spark engine's capabilities to perform advanced optimizations like predicate pushdown, constant folding, and expression simplification, leading to improved query performance.

Integration with Data Sources –

Spark SQL supports a wide range of data sources, including Parquet, Avro, ORC, JSON, and more. It allows users to read data from and write data to various storage systems and file formats. This flexibility makes it easier to work with diverse data sources in a unified manner.

Schema Inference and Evolution: Spark SQL can automatically infer the schema of structured data, eliminating the need for users to explicitly define the schema in many cases. Additionally, it supports schema evolution, allowing users to evolve the schema of their data over time without requiring changes to the existing data.



Interactive Querying: Spark SQL enables interactive querying of data using SQL, making it easier for users who are familiar with SQL to explore and analyze large datasets. This is particularly beneficial for analysts and data scientists who may not be proficient in traditional Spark programming.

Integration with Spark Ecosystem: Spark SQL seamlessly integrates with other Spark components and libraries, such as Spark Streaming for real-time data processing, MLlib for machine learning, and GraphX for graph processing. This integration allows users to build end-to-end data processing pipelines using a unified platform.

Optimized Execution Engine: Spark SQL uses the Tungsten execution engine, which is designed for in-memory data processing and code generation. This engine enhances the performance of Spark SQL queries by optimizing the execution plan and minimizing the data movement across the cluster.

Spark – SQL

Sample Program to read CSV Data –

#program

input :

eid,ename,sal,did

1,karan,25000,10

2,rajesh,35000,20

3,ganesh,15000,10

Code -

#read CSV



```
from pyspark.sql import SparkSession
spark=SparkSession.builder.appName("Spark SQL Test").getOrCreate()
#read
df=spark.read.format("CSV").option("header","true")\
    .load("file:///E:/emp.csv")
#show is action to show result from df default it shows 20 records
df.show()
```

Sample program with different delimiter and infer schema –

default delimiter = ','

delimiter = '|'

inferschema : it will identify data type of each field

ex-

```
#all req lib import
from pyspark.sql import SparkSession
spark=SparkSession.builder.appName("Spark SQL Test").getOrCreate()

#read
df=spark.read.format("CSV").option("header","true").\
    option("delimiter", "|")\
    .option("inferschema", "true")\
    .load("file:///E:/emp.csv")
```



#show is action to show result from df default it shows 20 records

```
df.show()
```

```
df.printSchema()
```

To Process data in Spark we have 2 Approach –

1. Pure SQL
2. DF API / DSL (Domain Specific Language) Approach –

#A. Pure SQL

In pure SQL we need to perform following steps –

- a. read data create df
- b. create temp view
- c. run queries or transformation
- d. save result

ex1.

```
#all req lib import
from pyspark.sql import SparkSession
spark=SparkSession.builder.appName("Spark SQL
Test").getOrCreate()
```

```
#read
```

```
df=spark.read.format("CSV").option("header","true").\
option("delimiter",":").\
.option("inferSchema","true").\
.load("file:///E:/emp.csv")
```



```
#create temp view in spark
df.createOrReplaceTempView("emp")
```

```
#sql queries on temp view
df1=spark.sql("select * from emp where sal >= 20000")
```

```
#show result from df1
df1.show()
```

```
#write into LFS
df1.write.format("CSV").option("header","true").\
    option("delimiter","-").save("file:///E:/sql_op")
```

ex2.

```
from pyspark.sql import SparkSession
spark=SparkSession.builder.appName("Sample").getOrCreate()
```

```
#read
df_emp=spark.read.format("CSV").option("header","true")\
    .option("delimiter",":").option("inferSchema","true")\
    .load("file:///E:/emp.csv")
```

```
df_emp.show()
```

```
df_dept=spark.read.format("CSV").option("header","true")\
    .option("delimiter",":").option("inferSchema","true")\
    .load("file:///E:/dept.csv")
```

```
df_dept.show()
```

```
df_emp.createOrReplaceTempView("emp")
df_dept.createOrReplaceTempView("dept")
```

```
df_res=spark.sql("""select eid,ename,dname from emp inner join dept
on
dept.did=emp.did""")
```

```
df_res.show()
```



```
df_res.write.save("file:///E:/empdept")  
df_res.write.format("CSV").save("file:///E:/empdeptcsv")
```

#B. DF API Approach (DSL)-

1. Pure SQL

In pure SQL approach to process data in spark we need to create temp view then we can

run sql queries on temp view to process data using pure SQL query.

```
df.createOrReplaceTempView("temp_table")  
df_res=spark.sql("select col from temp_table where col=<value>")  
df_res.show()
```

ex-

```
from pyspark.sql import SparkSession  
spark=SparkSession.builder.appName("Sample").getOrCreate()  
df=spark.read.format("CSV").option("delimiter","|")\  
    .option("header","true").option("inferSchema","true")\  
    .load("file:///E:/emp.csv")
```

```
#df.printSchema()
```

```
#df.show()
```

```
#process - pure sql
```

```
df.createOrReplaceTempView("emp")
```



```
df1=spark.sql("select * from emp where did=10")
df1.createOrReplaceTempView("emp1")
df2=spark.sql("select * from emp1 where sal > 30000")
df2.show()
df2.write.format("CSV").save("file:///E:/empop")
```

2. DF API / DSL

In DF API approach we no need to create temp view.

sh here we can run DF API functions Dataframe to process data.

we mostly use DF API approach in realtime.

ex.

```
from pyspark.sql import SparkSession
spark=SparkSession.builder.appName("Sample").getOrCreate()
```

```
df=spark.read.format("CSV").option("delimiter", "|").\
    option("header", "true").option("inferSchema", "true").\
    load("file:///E:/emp.csv")
df1=df.where(df['did']==10)
df2=df1.where(df['sal']>30000)
df2.show()
df2.write.format("CSV").save("file:///E:/empop1")
```

=====



DF API - functions

```
df=spark.read.format("CSV").option("delimiter", "|").\n    option("header", "true").option("inferSchema", "true")\n    .load("file:///E:/emp.csv")
```

- ✓ **select** : Selects specified columns or all columns.

```
df.select("*")\ndf.select("eid", "ename")\ndf1=df.select("eid", "ename")\ndf1.show()\n\ndf.select("eid", "ename").show()\ndf.select(df["eid"], df["ename"]).show()\ndf.select(df.eid, df.ename).show()\ndf.select(col("eid"), col("ename")).show()
```

```
from pyspark.sql.functions import *\nfrom pyspark.sql.functions import col
```

- ✓ **alias()** : Creates an alias (alternate name) for a column in a DataFrame.

```
df.select(df["eid"].alias("emp_id"), df["ename"].alias("emp_name")).show()
```

where/filter



where(condition): Filters rows in a DataFrame based on the specified condition. It is synonymous with the filter function.

```
df.filter(df['did']==10).show()
```

```
df.where(df['did']==10).show()
```

***filter on multiple condition**

```
df.filter((df['did']==10) & (df['sal']>30000)).show()
```

```
df.filter((df['did']==10) | (df['sal']>30000)).show()
```

#we can use multiple functions at a time using . operator

```
df.filter((df['did']==10) & (df['sal']>30000)).select(df['eid'],df['ename']).show()
```

withColumn("col_name",value)

withColumn(colName, col): Adds a new column or replaces an existing column with the specified expression in a DataFrame.

```
from pyspark.sql.functions import *
```

```
select eid,ename,did,1000 as bonus from emp;
```

```
df.show()
```



Sayu Softtech Training Institute

जहाँ लक्ष्य पुरा करने का सपना है

Training | Placement | Software solution | IT Project | JOb Support

```
df.withColumn("bonus",lit(1000)).show()
```

```
+---+-----+---+-----+-----+
```

```
|eid| ename|did| sal|bonus|
```

```
+---+-----+---+-----+-----+
```

```
| 1| rohan| 10|25000| 1000|
```

```
| 2|rakesh| 10|35000| 1000|
```

```
| 3| rani| 20|45000| 1000|
```

```
| 4|rajesh| 30|55000| 1000|
```

```
+---+-----+---+-----+-----+
```

```
df.withColumn("city",lit('Ahmednagar')).show()
```

```
+---+-----+---+-----+-----+
```

```
|eid| ename|did| sal| city|
```

```
+---+-----+---+-----+-----+
```

```
| 1| rohan| 10|25000|Ahmednagar|
```

```
| 2|rakesh| 10|35000|Ahmednagar|
```

```
| 3| rani| 20|45000|Ahmednagar|
```

```
| 4|rajesh| 30|55000|Ahmednagar|
```

```
+---+-----+---+-----+-----+
```

```
df.withColumn("today",current_date()).show()
```

```
+---+-----+---+-----+-----+
```



```
|eid|ename|did| sal|    today|
```

```
+---+-----+---+----+-----+
```

```
| 1| rohan| 10|25000|2024-02-03|
```

```
| 2|rakesh| 10|35000|2024-02-03|
```

```
| 3| rani| 20|45000|2024-02-03|
```

```
| 4|rajesh| 30|55000|2024-02-03|
```

```
+---+-----+---+----+-----+
```

```
df.withColumn("today",current_timestamp()).show()
```

```
+---+-----+---+----+-----+
```

```
|eid|ename|did| sal|          today|
```

```
+---+-----+---+----+-----+
```

```
| 1| rohan| 10|25000|2024-02-03 12:16:...|
```

```
| 2|rakesh| 10|35000|2024-02-03 12:16:...|
```

```
| 3| rani| 20|45000|2024-02-03 12:16:...|
```

```
| 4|rajesh| 30|55000|2024-02-03 12:16:...|
```

```
+---+-----+---+----+-----+
```

```
df.withColumn("today",current_timestamp()).show(20,False)
```

```
+---+-----+---+----+-----+
```

```
|eid|ename |did|sal |today          |
```

```
+---+-----+---+----+-----+
```

```
|1 |rohan |10 |25000|2024-02-03 12:17:27.46|
```

```
|2 |rakesh|10 |35000|2024-02-03 12:17:27.46|
```



|3 |rani |20 |45000|2024-02-03 12:17:27.46|

|4 |rajesh|30 |55000|2024-02-03 12:17:27.46|

+---+-----+---+-----+-----+

when otherwise

when(condition, value): Evaluates a condition and returns a value if true. Used in conjunction with the otherwise function to provide an alternative value if the condition is false in a DataFrame.

```
selece eid,ename,did,sal,
```

```
case
```

```
when did=10 then 1000
```

```
when did=20 then 2000
```

```
else 500
```

```
end as bonus from emp;
```

```
df.withColumn("bonus",when(df['did']==10,lit(1000)).when(df['did']==20,lit(2000)).otherwise(lit(500))).show()
```

+---+-----+---+-----+-----+

|eid| ename|did| sal|bonus|

+---+-----+---+-----+-----+

| 1| rohan| 10|25000| 1000|

| 2|rakesh| 10|35000| 1000|

| 3| rani| 20|45000| 2000|



```
| 4|rajesh| 30|55000| 500|
```

```
+---+-----+---+-----+-----+
```

withColumnRenamed("old","new")

withColumnRenamed(existing, new): Renames a column in a DataFrame from the existing name to the new name.

```
df.withColumnRenamed("sal","salary").show()
```

```
+---+-----+---+-----+
```

```
|eid| ename|did|salary|
```

```
+---+-----+---+-----+
```

```
| 1| rohan| 10| 25000|
```

```
| 2|rakesh| 10| 35000|
```

```
| 3| rani| 20| 45000|
```

```
| 4|rajesh| 30| 55000|
```

```
+---+-----+---+-----+
```

isNull

isNull(col): Checks if the values in a specified column are null, returning a boolean column indicating null or non-null values.

```
df=spark.read.format("CSV").option("delimiter","|")  
.option("header","true").option("inferSchema","true") .load("file:///E:/emp.csv")
```



```
df.filter(df['did'].isNull()).show()
```

```
+---+-----+---+-----+
|eid|ename| did|  sal|
+---+-----+---+-----+
|  5|Rajat|null|65000|
+---+-----+---+-----+
```

isNotNull

isNotNull(col): Checks if the values in a specified column are not null, returning a boolean column indicating non-null or null values.

```
df.filter(df['did'].isNotNull()).show()
```

```
+---+-----+---+-----+
|eid| ename|did|  sal|
+---+-----+---+-----+
|  1| rohan| 10|25000|
|  2|rakesh| 10|35000|
|  3| rani| 20|45000|
|  4|rajesh| 30|55000|
+---+-----+---+-----+
df.filter(~(df['did'].isNull())).show()
+---+-----+---+-----+
|eid| ename|did|  sal|
+---+-----+---+-----+
|  1| rohan| 10|25000|
```



```
| 2|rakesh| 10|35000|
```

```
| 3| rani| 20|45000|
```

```
| 4|rajesh| 30|55000|
```

```
+---+-----+---+-----+
```

distinct

distinct(): Returns a new DataFrame with distinct rows, removing duplicates.

```
eid|ename
```

```
1|rohan
```

```
2|rakesh
```

```
1|Rahul
```

```
1|Rajat
```

```
1|Rajat
```

```
1|Rajat
```

```
1|Rajat
```

```
>>> df.distinct().show()
```

```
+---+-----+
```

```
|eid| ename|
```

```
+---+-----+
```

```
| 1| Rahul|
```

```
| 1| Rajat|
```

```
| 2|rakesh|
```



```
| 1| rohan|
```

```
+---+-----+
```

dropDuplicates

dropDuplicates(): Returns a new DataFrame with duplicate rows removed based on the specified subset of columns or all columns if subset is not

```
>>> df.dropDuplicates().show()
```

```
+---+-----+
```

```
|eid| ename|
```

```
+---+-----+
```

```
| 1| Rahul|
```

```
| 1| Rajat|
```

```
| 2|rakesh|
```

```
| 1| rohan|
```

```
+---+-----+
```

```
>>> df.dropDuplicates(["eid"]).show()
```

```
+---+-----+
```

```
|eid| ename|
```

```
+---+-----+
```

```
| 1| rohan|
```

```
| 2|rakesh|
```

```
+---+-----+
```



Diffrence between distinct and dropDuplicates

distinct() and dropDuplicates() are both used to obtain unique/distinct rows from a DataFrame, but they have subtle differences:

distinct():

Functionality: Returns all unique rows in the DataFrame.

Usage: Doesn't take any arguments.

Result: Returns all unique rows considering all columns in the DataFrame.

```
df_distinct = df.distinct()
```

dropDuplicates(subset=None):

Functionality: Returns unique rows based on a subset of columns or all columns if subset is not provided.

Usage: Optionally, you can specify a subset of columns to consider for identifying duplicates.

Result: Returns unique rows based on the specified subset of columns or all columns if subset is not provided.

```
df_unique = df.dropDuplicates(subset=["col1", "col2"])
```

dropna

dropna(): Removes rows with null values in a DataFrame, allowing customization based on the how parameter (default is 'any') and an optional subset of columns.

+--+-----+--+-----+



```
|eid| ename| did| sal|
```

```
+---+-----+---+-----+
```

```
| 1| rohan| 10|25000|
```

```
| 2|rakesh| 10|35000|
```

```
| 3| null| 20|45000|
```

```
| 4|rajesh| 30|55000|
```

```
| 5| Rajat|null|65000|
```

```
+---+-----+---+-----+
```

```
>>> df.dropna().show()
```

```
+---+-----+---+-----+
```

```
|eid| ename|did| sal|
```

```
+---+-----+---+-----+
```

```
| 1| rohan| 10|25000|
```

```
| 2|rakesh| 10|35000|
```

```
| 4|rajesh| 30|55000|
```

```
+---+-----+---+-----+
```

fillna

fillna(): Returns a new DataFrame with null values replaced by the specified value, optionally restricted to a subset of columns

```
df.fillna('NotAvailable').show()
```

```
+---+-----+---+-----+
```

```
|eid| ename| did| sal|
```



```
+---+-----+---+----+
| 1|   rohan| 10|25000|
| 2|   rakesh| 10|35000|
| 3|NotAvailable| 20|45000|
| 4|   rajesh| 30|55000|
| 5|   Rajat|null|65000|
+---+-----+---+----+
```

```
df.fillna(0).show()
```

```
+---+-----+---+----+
|eid|  ename|did|  sal|
+---+-----+---+----+
| 1| rohan| 10|25000|
| 2|rakesh| 10|35000|
| 3|  null| 20|45000|
| 4|rajesh| 30|55000|
| 5| Rajat| 0|65000|
+---+-----+---+----+
```

```
df.fillna('NotAvailable').fillna(0).show()
```

```
+---+-----+---+----+
|eid|   ename|did|  sal|
+---+-----+---+----+
| 1|   rohan| 10|25000|
| 2|   rakesh| 10|35000|
```



```
| 3|NotAvailable| 20|45000|
```

```
| 4|    rajesh| 30|55000|
```

```
| 5|    Rajat| 0|65000|
```

```
+---+-----+---+---+
```

groupBy

groupBy(*cols): Groups the DataFrame by the specified columns, enabling the application of aggregate functions on each group.

```
>>> df.groupBy("did").count().show()
```

```
+---+-----+
```

```
| did|count|
```

```
+---+-----+
```

```
|null|    1|
```

```
| 20|    1|
```

```
| 10|    2|
```

```
| 30|    1|
```

```
+---+-----+
```

```
>>> df.groupBy("did").max("sal").show()
```

```
+---+-----+
```

```
| did|max(sal)|
```

```
+---+-----+
```

```
|null| 65000|
```

```
| 20| 45000|
```

```
| 10| 35000|
```



```
| 30| 55000|
```

```
+---+-----+
```

```
>>> df.groupby("did").min("sal").show()
```

```
+---+-----+
```

```
| did|min(sal)|
```

```
+---+-----+
```

```
|null| 65000|
```

```
| 20| 45000|
```

```
| 10| 25000|
```

```
| 30| 55000|
```

```
+---+-----+
```

```
>>> df.groupby("did").sum("sal").show()
```

```
+---+-----+
```

```
| did|sum(sal)|
```

```
+---+-----+
```

```
|null| 65000|
```

```
| 20| 45000|
```

```
| 10| 60000|
```

```
| 30| 55000|
```

```
+---+-----+
```

```
>>> df.groupby("did").avg("sal").show()
```

```
+---+-----+
```



```
| did|avg(sal)|
```

```
+----+-----+
```

```
|null| 65000.0|
```

```
| 20| 45000.0|
```

```
| 10| 30000.0|
```

```
| 30| 55000.0|
```

```
+----+-----+
```

groupByHaving

```
df.groupBy("did").sum("sal").filter(df["did"]==10).show()
```

group functions

agg(exprs): Aggregates the data in a DataFrame using specified aggregate expressions.

```
>>> df.agg(max("sal")).show()
```

```
+-----+
```

```
|max(sal)|
```

```
+-----+
```

```
| 65000|
```

```
+-----+
```



```
>>> df.agg(min("sal")).show()
```

```
+-----+
```

```
|min(sal)|
```

```
+-----+
```

```
| 25000|
```

```
+-----+
```

```
>>> df.agg(sum("sal")).show()
```

```
+-----+
```

```
|sum(sal)|
```

```
+-----+
```

```
| 225000|
```

```
+-----+
```

```
>>> df.agg(avg("sal")).show()
```

```
+-----+
```

```
|avg(sal)|
```

```
+-----+
```

```
| 45000.0|
```

```
+-----+
```

count

count(): Computes the number of rows in a DataFrame.

```
>>> df.count()
```



```
df=spark.read.format("CSV").option("header","true").option("inferSchema","true").load("file:///D:/emp.txt")
```

drop()

drop(*cols): Returns a new DataFrame with specified columns removed.

```
>>> df.drop("sal").show()
```

```
+---+-----+---+
|eid|  ename|did|
+---+-----+---+
| 1| karan| 10|
| 2|rajesh| 10|
| 3|ganesh| 20|
| 4|rakesh| 30|
| 5|  rani| 30|
| 6|  roha| 30|
| 7|mayank| 40|
+---+-----+---+
```

```
>>> df.drop("sal","did").show()
```

```
+---+-----+
|eid|  ename|
+---+-----+
```



+---+-----+

1	karan
2	rajesh
3	ganesh
4	rakesh
5	rani
6	roha
7	mayank

df.printSchema()

dtypes

dtypes: Returns a list of tuples representing the names and data types of all columns in a DataFrame.

df.dtypes

[('eid', 'int'), ('ename', 'string'), ('did', 'int'), ('sal', 'int')]

join

join(other, on=None, how=None): Joins two DataFrames based on a common column or expression, with an optional specified join type (how).

```
df_emp=spark.read.format("CSV").option("header","true").option("inferSchema","true")  
.load("file:///D:/emp.txt")
```



```
df_dept=spark.read.format("CSV").option("header","true").option("inferSchema","true")
).load("file:///D:/dept.txt")
```

```
df_emp.show()
```

```
df_dept.show()
```

```
df_emp.join(df_dept,"did","inner").show()
```

```
>>> df_emp.join(df_dept,"did","inner").show()
```

```
+---+---+-----+-----+-----+
```

```
|did|eid| ename| sal|dname|
```

```
+---+---+-----+-----+-----+
```

```
| 10| 1| karan|25000| IT|
```

```
| 10| 2|rajesh|35000| IT|
```

```
| 20| 3|ganesh|45000| RD|
```

```
| 30| 4|rakesh|56000|Sales|
```

```
| 30| 5| rani|56000|Sales|
```

```
| 30| 6| roha|56000|Sales|
```

```
+---+---+-----+-----+-----+
```

```
df_emp.join(df_dept,"did","inner").select(df_emp["*"],df_dept["dname"]).show()
```

```
df_emp.join(df_dept,"did","left").show()
```

```
+---+---+-----+-----+-----+
```



Sayu Softtech Training Institute

जहाँ आपके प्रोग्राम केवल सच है

Training | Placement | Software solution | IT Project | JOB Support

```
|did|eid| ename| sal|dname|
```

```
+---+---+-----+-----+-----+
```

```
| 10| 1| karan|25000| IT|
```

```
| 10| 2|rajesh|35000| IT|
```

```
| 20| 3|ganesh|45000| RD|
```

```
| 30| 4|rakesh|56000|Sales|
```

```
| 30| 5| rani|56000|Sales|
```

```
| 30| 6| roha|56000|Sales|
```

```
| 70| 7|mayank|67000| null|
```

```
+---+---+-----+-----+-----+
```

```
>>> df_emp.join(df_dept,"did","right").show()
```

```
+---+---+-----+-----+-----+
```

```
|did| eid| ename| sal|dname|
```

```
+---+---+-----+-----+-----+
```

```
| 10| 2|rajesh|35000| IT|
```

```
| 10| 1| karan|25000| IT|
```

```
| 20| 3|ganesh|45000| RD|
```

```
| 30| 6| roha|56000|Sales|
```

```
| 30| 5| rani|56000|Sales|
```

```
| 30| 4|rakesh|56000|Sales|
```

```
| 40|null| null| null| Adv|
```

```
| 50|null| null| null| DM|
```

```
+---+---+-----+-----+-----+
```



```
>>> df_emp.join(df_dept,"did","full").show()
```

```
+---+---+---+---+---+
|did|eid|ename|sal|dname|
+---+---+---+---+---+
| 20| 3|ganesh|45000| RD|
| 40|null| null| null| Adv|
| 10| 1|karan|25000| IT|
| 10| 2|rajesh|35000| IT|
| 50|null| null| null| DM|
| 70| 7|mayank|67000| null|
| 30| 4|rakesh|56000|Sales|
| 30| 5| rani|56000|Sales|
| 30| 6| roha|56000|Sales|
+---+---+---+---+---+
```

```
>>> df_emp.crossJoin(df_dept).show()
```

```
+---+---+---+---+---+
|eid|ename|did|sal|did|dname|
+---+---+---+---+---+
| 1|karan| 10|25000| 10| IT|
| 1|karan| 10|25000| 20| RD|
| 1|karan| 10|25000| 30|Sales|
| 1|karan| 10|25000| 40| Adv|
| 1|karan| 10|25000| 50| DM|
| 2|rajesh| 10|35000| 10| IT|
```



```
| 2|rajesh| 10|35000| 20| RD|
| 2|rajesh| 10|35000| 30|Sales|
| 2|rajesh| 10|35000| 40| Adv|
| 2|rajesh| 10|35000| 50| DM|
| 3|ganesh| 20|45000| 10| IT|
| 3|ganesh| 20|45000| 20| RD|
| 3|ganesh| 20|45000| 30|Sales|
| 3|ganesh| 20|45000| 40| Adv|
| 3|ganesh| 20|45000| 50| DM|
| 4|rakesh| 30|56000| 10| IT|
| 4|rakesh| 30|56000| 20| RD|
| 4|rakesh| 30|56000| 30|Sales|
| 4|rakesh| 30|56000| 40| Adv|
| 4|rakesh| 30|56000| 50| DM|
```

```
+---+-----+---+-----+---+-----+
```

only showing top 20 rows

```
>> df_emp.join(df_dept,"did","left_anti").show()
```

```
+---+-----+---+-----+
```

```
|did|eid| ename| sal|
```

```
+---+-----+---+-----+
```

```
| 70| 7|mayank|67000|
```

```
+---+-----+---+-----+
```

```
>>> df_dept.join(df_emp,"did","left_anti").show()
```



+---+-----+

|did|dname|

+---+-----+

| 40| Adv|

| 50| DM|

+---+-----+

Set Operator

#union (spark) = union all (sql)

union(other): Concatenates two DataFrames vertically, stacking the rows of one DataFrame on top of the other. The DataFrames must have the same schema.

```
df1=spark.read.format("CSV").option("header","true").option("inferSchema","true").load("file:///D:/emp.txt")
```

```
df2=spark.read.format("CSV").option("header","true").option("inferSchema","true").load("file:///D:/emp1.txt")
```

```
>>> df1.union(df2).show()
```

+---+-----+---+-----+

|eid| ename|did| sal|

+---+-----+---+-----+

| 1| karan| 10|25000|

| 2| rajesh| 10|35000|

| 3| ganesh| 20|45000|



Sayu Softtech Training Institute

जहाँ लक्ष्य पुरा करने का सपना है

Training | Placement | Software solution | IT Project | JOB Support

```
| 4| rakesh| 30|56000|
| 5| rani| 30|56000|
| 6| roha| 30|56000|
| 7| mayank| 70|67000|
| 1| karan| 10|25000|
| 2| rajesh| 10|35000|
| 3| ganesh| 20|45000|
| 4| rakesh| 30|56000|
| 8| Neel| 30|56000|
| 9|Gurpreet| 30|56000|
| 10| Peter| 70|67000|
```

```
+---+-----+---+-----+
```

```
>>> df1.union(df2).distinct().show()
```

```
+---+-----+---+-----+
```

```
|eid|  ename|did|  sal|
```

```
+---+-----+---+-----+
```

```
| 2| rajesh| 10|35000|
| 7| mayank| 70|67000|
| 10| Peter| 70|67000|
| 4| rakesh| 30|56000|
| 6| roha| 30|56000|
| 8| Neel| 30|56000|
| 3| ganesh| 20|45000|
| 9|Gurpreet| 30|56000|
```



```
| 1| karan| 10|25000|
```

```
| 5| rani| 30|56000|
```

```
+---+-----+---+-----+
```

#intersect:

intersect(other): Returns a new DataFrame containing rows that exist in both the original DataFrame and the specified other DataFrame, removing duplicate rows. The DataFrames must have the same schema.

```
>>> df1.intersect(df2).show()
```

```
+---+-----+---+-----+
```

```
|eid| ename|did|  sal|
```

```
+---+-----+---+-----+
```

```
| 2|rajesh| 10|35000|
```

```
| 4|rakesh| 30|56000|
```

```
| 3|ganesh| 20|45000|
```

```
| 1| karan| 10|25000|
```

```
+---+-----+---+-----+
```

#exceptAll:

exceptAll(other): Returns a new DataFrame with rows from the original DataFrame that do not exist in the specified other DataFrame. It retains all occurrences of rows, including duplicates. The DataFrames must have the same schema.

```
>>> df1.exceptAll(df2).show()
```

```
+---+-----+---+-----+
```

```
|eid| ename|did|  sal|
```



```
+---+-----+---+-----+
```

```
| 7|mayank| 70|67000|
```

```
| 6| roha| 30|56000|
```

```
| 5| rani| 30|56000|
```

```
+---+-----+---+-----+
```

```
>>> df2.exceptAll(df1).show()
```

```
+---+-----+---+-----+
```

```
|eid|  ename|did| sal|
```

```
+---+-----+---+-----+
```

```
| 10|  Peter| 70|67000|
```

```
| 8|   Neel| 30|56000|
```

```
| 9|Gurpreet| 30|56000|
```

```
+---+-----+---+-----+
```

Windowing functions

Windowing functions in PySpark are used for performing calculations over a specified range of rows related to the current row in a DataFrame. They are particularly useful for tasks that involve ranking, aggregation, and partitioning of data within a specific window. These functions are commonly used in conjunction with the `over` clause.

Here are some key points about windowing functions in PySpark:

Syntax:

Windowing functions are typically applied using the `over` clause, which takes a `WindowSpec` as an argument.



The WindowSpec can be defined using the Window class from the pyspark.sql.window module.

WindowSpec:

The WindowSpec defines the partitioning and ordering of rows within a window.

It includes specifications like partitionBy to define partition columns and orderBy to specify the sorting order

exm-

```
from pyspark.sql.window import Window
```

```
from pyspark.sql.functions import row_number
```

```
window_spec = Window.partitionBy("department").orderBy("salary")
```

```
df.withColumn("row_number", row_number().over(window_spec))
```

Common Windowing Functions:

row_number(): Assigns a unique number to each row within a partition.

rank(): Assigns a unique rank to each distinct row within a partition with tied rows receiving the same rank.

dense_rank(): Similar to rank(), but without gaps in the ranking sequence.

Aggregation functions like sum(), avg(), etc., can also be used as window functions.

lead(col, offset=1, default=None) and lag(col, offset=1, default=None) are window functions in PySpark used for accessing data in subsequent or preceding rows within a window, respectively.

lag(col, offset=1, default=None):



Returns the value of the specified column in the row that is offset number of rows before the current row.

If the offset goes beyond the first row of the window, it returns the specified default value or None if not provided.

```
eid,ename,did,sal
1,karan,10,25000
2,rajesh,10,35000
3,ganesh,20,45000
4,rakesh,30,56000
5,rani,30,56000
6,roha,30,56000
7,mayank,70,67000
```

```
df.show()
```

```
from pyspark.sql.functions import *
from pyspark.sql.window import Window
```

```
df.withColumn("rn",row_number().over(Window.partitionBy("did").orderBy("sal"))).show()
```

```
df.withColumn("rn",rank().over(Window.partitionBy("did").orderBy("sal"))).show()
```



```
df.withColumn("rn",dense_rank().over(Window.partitionBy("did").orderBy("sal"))).show()
```

```
window_spec=Window.partitionBy("did").orderBy("sal")
df.withColumn("rn",dense_rank().over(window_spec)).show()
```

```
#lead...lag
```

```
df.withColumn("next_sal",lead("sal",2,0).over(Window.orderBy("sal"))).show()
```

```
df.withColumn("next_sal",lag("sal",2,0).over(Window.orderBy("sal"))).show()
```

```
df.withColumn("next_sal",lag("sal",2,0).over(Window.orderBy(desc("sal")))).show()
```

Date Functions

```
current_date()
```

The `current_date()` function in PySpark retrieves the current date, and `df.withColumn("current_date", current_date())` adds a new column named "current_date" to the DataFrame, populating it with the current date for each row.

```
current_timestamp()
```

`current_timestamp()`: Returns the current timestamp as a PySpark DataFrame column. This function is used to create a new column representing the current timestamp in a DataFrame.



```
df1=spark.read.format("CSV").option("header","true").option("inferSchema","true").option("timestampFormat","yyyy-MM-dd HH:mm:ss").load("file:///D:/emp.txt")
```

```
df.withColumn("today",current_date()).show()
```

```
df.withColumn("today",current_timestamp()).show()
```

`#year(col)`: Extracts the year from a timestamp column, creating a new DataFrame column. Useful for temporal analysis.

`month(col)`: Extracts the month from a timestamp column, creating a new DataFrame column. Enhances temporal insights.

`dayofmonth(col)`: Extracts the day of the month from a timestamp column, adding a new DataFrame column. Aids in date-based analysis.

`hour(col)`: Extracts the hour from a timestamp column, generating a new DataFrame column. Useful for time-based exploration.

`minute(col)`: Extracts the minute from a timestamp column, adding a new DataFrame column. Enhances time-related details.

`second(col)`: Extracts the second from a timestamp column, creating a new DataFrame column. Useful for finer-grained time analysis.

`dayofweek(col)`: Extracts the day of the week from a timestamp column, adding a new DataFrame column. Facilitates weekday-based analysis.



```
df1.withColumn("new_col",year("doj")).show()
```

```
df1.withColumn("new_col",month("doj")).show()
```

```
>>> df1.withColumn("new_col",dayofmonth("doj")).show()
```

```
+--+-----+--+-----+-----+-----+
|id|  ename|did|  sal|      doj|new_col|
+--+-----+--+-----+-----+-----+
| 1| karan| 10|25000|2023-12-31 00:12:01|   31|
| 2|rajesh| 10|35000|2024-01-01 00:12:01|    1|
| 3|ganesh| 20|45000|2023-12-31 00:12:01|   31|
| 4|rakesh| 30|56000|2023-11-21 00:12:01|   21|
| 5| rani| 30|56000|2022-05-11 00:12:01|   11|
| 6| roha| 30|56000|2023-12-31 00:12:01|   31|
| 7|mayank| 30|67000|2024-02-01 00:12:01|    1|
+--+-----+--+-----+-----+-----+
```

```
>>> df1.withColumn("new_col",hour("doj")).show()
```

```
+--+-----+--+-----+-----+-----+
|id|  ename|did|  sal|      doj|new_col|
+--+-----+--+-----+-----+-----+
| 1| karan| 10|25000|2023-12-31 00:12:01|    0|
| 2|rajesh| 10|35000|2024-01-01 00:12:01|    0|
| 3|ganesh| 20|45000|2023-12-31 00:12:01|    0|
| 4|rakesh| 30|56000|2023-11-21 00:12:01|    0|
```



Sayu Softtech Training Institute

जहाँ लक्ष्य पुरस्कार है वास्तविकता

Training | Placement | Software solution | IT Project | JOb Support

```
| 5| rani| 30|56000|2022-05-11 00:12:01|    0|
| 6| roha| 30|56000|2023-12-31 00:12:01|    0|
| 7|mayank| 30|67000|2024-02-01 00:12:01|    0|
```

```
+---+-----+---+-----+-----+-----+
```

```
>>> df1.withColumn("new_col",minute("doj")).show()
```

```
+---+-----+---+-----+-----+-----+
```

```
|eid|  ename|did|  sal|          doj|new_col|
+---+-----+---+-----+-----+-----+
| 1| karan| 10|25000|2023-12-31 00:12:01|    12|
| 2|rajesh| 10|35000|2024-01-01 00:12:01|    12|
| 3|ganesh| 20|45000|2023-12-31 00:12:01|    12|
| 4|rakesh| 30|56000|2023-11-21 00:12:01|    12|
| 5| rani| 30|56000|2022-05-11 00:12:01|    12|
| 6| roha| 30|56000|2023-12-31 00:12:01|    12|
| 7|mayank| 30|67000|2024-02-01 00:12:01|    12|
```

```
+---+-----+---+-----+-----+-----+
```

```
>>> df1.withColumn("new_col",second("doj")).show()
```

```
+---+-----+---+-----+-----+-----+
```

```
|eid|  ename|did|  sal|          doj|new_col|
+---+-----+---+-----+-----+-----+
| 1| karan| 10|25000|2023-12-31 00:12:01|     1|
| 2|rajesh| 10|35000|2024-01-01 00:12:01|     1|
| 3|ganesh| 20|45000|2023-12-31 00:12:01|     1|
```



Sayu Softtech Training Institute

जहाँ लक्ष्य पुराण केवल है reality

Training | Placement | Software solution | IT Project | JOB Support

```
| 4|rakesh| 30|56000|2023-11-21 00:12:01|    1|
| 5| rani| 30|56000|2022-05-11 00:12:01|    1|
| 6| roha| 30|56000|2023-12-31 00:12:01|    1|
| 7|mayank| 30|67000|2024-02-01 00:12:01|    1|
```

```
+--+-----+--+-----+-----+-----+
```

```
>>> df1.withColumn("new_col",dayofweek("doj")).show()
```

```
+--+-----+--+-----+-----+-----+
```

```
|eid| ename|did|  sal|          doj|new_col|
+--+-----+--+-----+-----+-----+
| 1| karan| 10|25000|2023-12-31 00:12:01|    1|
| 2|rajesh| 10|35000|2024-01-01 00:12:01|    2|
| 3|ganesh| 20|45000|2023-12-31 00:12:01|    1|
| 4|rakesh| 30|56000|2023-11-21 00:12:01|    3|
| 5| rani| 30|56000|2022-05-11 00:12:01|    4|
| 6| roha| 30|56000|2023-12-31 00:12:01|    1|
| 7|mayank| 30|67000|2024-02-01 00:12:01|    5|
```

```
+--+-----+--+-----+-----+-----+
```

```
df1.withColumn("diff",datediff(current_timestamp(),("doj"))).show()
```



```
>> df1.withColumn("new_col",date_add("doj",1)).show()
```

```
+---+-----+---+-----+-----+-----+
|eid| ename|did|  sal|      doj|  new_col|
+---+-----+---+-----+-----+-----+
| 1| karan| 10|25000|2023-12-31 00:12:01|2024-01-01|
| 2|rajesh| 10|35000|2024-01-01 00:12:01|2024-01-02|
| 3|ganesh| 20|45000|2023-12-31 00:12:01|2024-01-01|
| 4|rakesh| 30|56000|2023-11-21 00:12:01|2023-11-22|
| 5| rani| 30|56000|2022-05-11 00:12:01|2022-05-12|
| 6| roha| 30|56000|2023-12-31 00:12:01|2024-01-01|
| 7|mayank| 30|67000|2024-02-01 00:12:01|2024-02-02|
+---+-----+---+-----+-----+-----+
```

```
>> df1.withColumn("new_col",date_sub("doj",5)).show()
```

```
+---+-----+---+-----+-----+-----+
|eid| ename|did|  sal|      doj|  new_col|
+---+-----+---+-----+-----+-----+
| 1| karan| 10|25000|2023-12-31 00:12:01|2023-12-26|
| 2|rajesh| 10|35000|2024-01-01 00:12:01|2023-12-27|
| 3|ganesh| 20|45000|2023-12-31 00:12:01|2023-12-26|
| 4|rakesh| 30|56000|2023-11-21 00:12:01|2023-11-16|
| 5| rani| 30|56000|2022-05-11 00:12:01|2022-05-06|
| 6| roha| 30|56000|2023-12-31 00:12:01|2023-12-26|
| 7|mayank| 30|67000|2024-02-01 00:12:01|2024-01-27|
+---+-----+---+-----+-----+-----+
```



===== **CSV ---> Process ----> CSV**

```
df1=spark.read.format("CSV").option("header","true").option("inferSchema","true").option("timestampFormat","yyyy-MM-dd HH:mm:ss").load("file:///D:/emp.txt")
```

```
df2=df1.filter(df1['did']==10)
```

```
df2.write.format("CSV").save("file:///D:/empop")
```

mode

```
.overwrite
```

```
.append
```

```
df2.write.format("CSV").mode("overwrite").save("file:///D:/empop")
```

```
df2.write.format("CSV").mode("append").save("file:///D:/empop")
```

```
#HW
```

```
dirA--> read --> process--> dirA write
```

=====



Write in Partition

```
df1=spark.read.format("CSV").option("header","true").option("inferSchema","true").option("timestampFormat","yyyy-MM-dd HH:mm:ss").load("file:///D:/emp.txt")
```

```
df1.write.format("CSV").partitionBy("did").save("file:///D:/empp")
```

=====

#read As CSV and write as Parquet

```
df1=spark.read.format("CSV").option("header","true").option("inferSchema","true").option("timestampFormat","yyyy-MM-dd HH:mm:ss").load("file:///D:/emp.txt")
```

```
df1.write.format("PARQUET").save("file:///D:/emp1")
```

```
df1.write.save("file:///D:/emp2")
```

=====

==

#Read Data as CSV and write in Json

```
df1=spark.read.format("CSV").option("header","true").option("inferSchema","true").option("timestampFormat","yyyy-MM-dd HH:mm:ss").load("file:///D:/emp.txt")
```

```
df1.write.format("JSON").save("file:///D:/emp3")
```



#Read Data as CSV and write in ORC

```
df1=spark.read.format("CSV").option("header","true").option("inferSchema","true").option("timestampFormat","yyyy-MM-dd HH:mm:ss").load("file:///D:/emp.txt")
```

```
df1.write.format("ORC").save("file:///D:/emp4")
```

=====

Any file format we can read and write

CSV --> Parquet, ORC, JSON

```
df1=spark.read.format("PARQUET").load("file:///D:/emp1/emp.parquet")
```

```
df1.show()
```

=====



Process Json Data using Pyspark -

In PySpark, the JSON data type refers to data that is stored in the JavaScript Object Notation (JSON) format. JSON is a lightweight data interchange format that is commonly used for storing and exchanging structured data between systems.

Structure:

JSON data consists of key-value pairs, where keys are strings and values can be strings, numbers, arrays, objects (nested JSON), boolean values, or null.

JSON can represent complex data structures with nested objects and arrays, providing a flexible way to organize and represent data.

Usage in PySpark:

In PySpark, JSON data is often used for semi-structured data, where the schema is not rigidly defined and may vary from one record to another.

JSON data is commonly processed in PySpark DataFrames, which provide a structured way to work with semi-structured data.

JSON data type in PySpark provides a flexible and convenient way to work with semi-structured data, enabling efficient processing and analysis of data stored in JSON format within a distributed computing environment.

Process simple json file –

json files :

<https://github.com/sagaringale7/Dataset/tree/main/json>

#simple Json - code

```
df=spark.read.format("JSON").load("file:///D:/sample.json")
df1=df.filter(df['ZipCodeType']=='STANDARD')
df1.show()
```



Note – By default spark consider json record should be in single line.

#Multiline json Processing :

```
from pyspark.sql import SparkSession

spark=SparkSession.builder.appName("Sample").getOrCreate()

df=spark.read.format("JSON").option("multiline","true").load("file:///D:/sample_mul.json")

df.show()
```

#Nested Json Processing

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import *

spark=SparkSession.builder.appName("Sample").getOrCreate()

df=spark.read.format("JSON").option("multiline","true").load("D:/persons.json")
```



```
df1=df.withColumn("persons",explode(df
["persons"])).\

withColumn("cars",explode("persons.car
s")).\

withColumn("models",explode("cars.mode
ls")).\

selectExpr("persons.name","persons.age
","cars.name as car_name","models")

df2=df1.groupBy("name").count()

df2.show()
```

Nested Json exp 2 - :

```
from pyspark.sql import SparkSession

from pyspark.sql.functions import *

spark=SparkSession.builder.appName("co
mplex json").getOrCreate()

df=spark.read.format("Json").option("m
ultiline","true").\
    load("file:///D:/nes.json")
```



```
df1=df.withColumn("source",explode(array("source.*"))).\
withColumn("id",col("source.id")).\
withColumn("ip",col("source.ip"))\
.withColumn("temp",col("source.temp"))\
.withColumn("description",col("source.description"))\
.withColumn("c02_level",col("source.c02_level"))\
.withColumn("lat",col("source.geo.lat"))\
.withColumn("long",col("source.geo.long"))\
df1.printSchema()

df1.show()
```



Process Data from RDBMS –

To process data from RDBMS in pyspark we need to use format at JDBC and provided all required details like url,username,password,dbtable,driver

Driver name will get change as per DBMS which we are referring.

Exm-

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import *

spark=SparkSession.builder.appName("RDBMS data process").getOrCreate()

#read data from RDBMS - mysql

df_emp=spark.read.format("JDBC").option("url","jdbc:mysql://database-1.cxau8dx2g7op.ap-south-1.rds.amazonaws.com:3306/DEV").\
    option("user","myuser").\
    option("password","mypassword").\
    option("dbtable","emp").\
    option("driver","com.mysql.cj.jdbc.Dri
```





```
load()

#read data for dept table
df_dept=spark.read.format("JDBC").option("url","jdbc:mysql://database-1.cxau8dx2g7op.ap-south-1.rds.amazonaws.com:3306/DEV").\
    option("user","myuser").\
    option("password","mypassword").\
    option("dbtable","dept").\

option("driver","com.mysql.cj.jdbc.Driver").\
    load()

df_emp.createOrReplaceTempView("e")
df_dept.createOrReplaceTempView("d")

df=spark.sql("select e.*,d.dname from e inner join d on e.did=d.did")

#writing in RDBMS
df.write.format("JDBC").option("url","jdbc:mysql://database-1.cxau8dx2g7op.ap-south-1.rds.amazonaws.com:3306/DEV").\
```



```
option("user", "myuser").\  
option("password", "mypassword").\  
option("dbtable", "emp_dept").\  
  
option("driver", "com.mysql.cj.jdbc.Dri  
ver").\  
save()
```

exm3 – instead of dbtable we can give custom query as well

```
from pyspark.sql import SparkSession  
from pyspark.sql.functions import *  
  
spark=SparkSession.builder.appName("RD  
BMS data process").getOrCreate()  
  
#read data from RDBMS - mysql  
  
df_emp=spark.read.format("JDBC").optio  
n("url", "jdbc:mysql://database-  
1.cxau8dx2g7op.ap-south-  
1.rds.amazonaws.com:3306/DEV").\  
option("user", "myuser").\  
option("password", "mypassword").\  
option("query", "select * from emp  
where city='Pune'").\  
  
option("driver", "com.mysql.cj.jdbc.Dri
```



```
ver") .\
    load()

#show data

df_emp.show()
```

Process Data from S3 or HDFS or Hive –

Like LFS we can give path for HDFS and S3 to process data.

To Read or write data in Hive we need to enableHiveSupport() while create Spark Session Object.

#HDFS to Hive

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import *

spark=SparkSession.builder.appName("HDFS to Hive")\
    .enableHiveSupport().getOrCreate()

df=spark.read.format("CSV").option("header","true").\

option("inferSchema","true").load("/cust.csv")
```



```
df.show()

df.write.saveAsTable("prod.cust")
```

#Hive To S3

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import *

spark=SparkSession.builder.appName("RD
BMS data process")\
    .enableHiveSupport().getOrCreate()

df=spark.sql("select * from
prod.cust_hemant where city='pune'")

df.show()

df.write.format("json").save("s3://cla
ssusa/output/cust_hemant")
```

Process Data from Hbase –

```
#Hbase to HDFS

from pyspark.sql import SparkSession
from pyspark.sql.functions import *
```



```
spark=SparkSession.builder.master("yarn")\
    .appName("sample").getOrCreate()

#read from Hbase
df=spark.read.format("org.apache.phoenix.spark")\
    .option("zkUrl","localhost:2181")\
    .option("table","CUST").load()

#process
df1=df.withColumn("data_processed_dt",current_timestamp())

#load
df1.write.format("CSV").option("header","true")\
    .option("delimiter","|").save("/hb_op")

print("Job Successfully completed")
```

S3 to Hbase :

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import *

spark=SparkSession.builder.appName("sample")\
    .getOrCreate()
```



```
#read from s3

df=spark.read.format("CSV").option("header","true").option("inferSchema","true").load("s3://b12class/input_data/loan-test.csv")

df.show()

df1=df.filter(df["Education"]=='Graduate')

df1.write.format("org.apache.phoenix.spark").option("zkUrl","localhost:2181").option("table","loan").mode('overwrite').save()

print("Data successfully written in Hbase")
```

Job Run Commands –

```
spark-submit --jars /usr/lib/phoenix/phoenix-4.14.3-HBase-1.4-client.jar
Varchar(20),/usr/lib/phoenix/phoenix-spark-4.14.3-HBase-1.4.jar --master yarn --
driver-memory 1G --executor-memory 3G s3tohbase.py
```



custom schema -

- ✓ Defining a custom schema in PySpark allows precise control over the structure of DataFrames.
- ✓ The `pyspark.sql.types` module provides tools for constructing custom schemas.
- ✓ A schema consists of fields, each defined by a name, data type, and nullability.
- ✓ `StructType` is used to define the overall structure, while `StructField` specifies individual fields.
- ✓ DataFrames created with custom schemas enforce data integrity by ensuring adherence to specified field types and nullability constraints.
- ✓ This approach is particularly beneficial when working with structured data where consistency is paramount.
- ✓ Custom schemas facilitate seamless integration with external data sources and enable efficient data manipulation and analysis.
- ✓ They enhance code readability and maintainability by explicitly documenting the expected structure of the data.
- ✓ Utilizing custom schemas promotes data quality and reliability in PySpark applications

Exm –

```
from pyspark.sql import SparkSession
from pyspark.sql.types import *

spark=SparkSession.builder.appName("Without header").getOrCreate()

header=StructType([

StructField("eid",IntegerType(),True),
```



```
StructField("ename", StringType(), True),  
,  
StructField("did", IntegerType(), True),  
StructField("sal", IntegerType(), True),  
StructField("city", StringType(), True)  
])  
  
#read  
df=spark.read.format("CSV").option("inferSchema", "true") \  
    .schema(header) \  
    .load("D:/emp.csv")  
  
df.show()
```

Handling bad records -

Handling bad records, such as invalid or malformed data, is crucial in data processing pipelines to ensure data quality and prevent job failures. PySpark provides several mechanisms for dealing with bad records:

FAILFAST Mode: FAILFAST mode is the most restrictive. It immediately raises an exception and fails the entire job if it encounters any corrupt records during the reading process. This mode is suitable when you want to ensure strict data integrity and prefer to abort the job rather than risk processing potentially unreliable data.



DROPMALFORMED Mode: In DROPMALFORMED mode, PySpark simply drops the rows that contain corrupt records during the reading process. It discards any rows that cannot be correctly parsed according to the specified schema. This mode is suitable when you prioritize data quality over quantity and can afford to discard potentially valuable information.

PERMISSIVE Mode: In this mode, PySpark tries to parse all records it can and sets null for corrupted records. It does not raise an exception or fail the job. Instead, it logs the corrupt records into a specified column, which can be later inspected. This mode is useful when you want to process as much data as possible while still capturing and handling the corrupted records.

1. FASTFAIL –

```
• from pyspark.sql import
  SparkSession
  from pyspark.sql.types import *

  spark=SparkSession.builder.appName(
    "Without header").getOrCreate()

  header=StructType([

    StructField("eid", IntegerType(), True),

    StructField("ename", StringType(), True),

    StructField("did", IntegerType(), True)
```



```
e) ,  
  
StructField("sal", IntegerType(), True)  
e) ,  
  
StructField("city", StringType(), True)  
e)  
])  
  
#read  
df=spark.read.format("CSV").option(  
    "inferSchema", "true") \  
    .schema(header) \  
    .option("mode", "FAILFAST") \  
    .load("D:/emp.csv")  
  
df.show()
```

2. DROPMALFORMED –

```
from pyspark.sql import SparkSession  
from pyspark.sql.types import *  
  
spark=SparkSession.builder.appName("With  
out header").getOrCreate()  
  
header=StructType([
```



```
StructField("eid", IntegerType(), True),
StructField("ename", StringType(), True)
',
StructField("did", IntegerType(), True),
StructField("sal", IntegerType(), True),
StructField("city", StringType(), True)
])

#read
df=spark.read.format("CSV").option("inferSchema", "true") \
    .schema(header) \
    .option("mode", "DROPMALFORMED") \
    .load("D:/emp.csv")

df.show()
```

3. PERMISSIVE –

```
from pyspark.sql import SparkSession
from pyspark.sql.types import *

spark=SparkSession.builder.appName("Without header").getOrCreate()
```



```
header=StructType([
    StructField("eid", IntegerType(), True),
    StructField("ename", StringType(), True),
    StructField("did", IntegerType(), True),
    StructField("sal", IntegerType(), True),
    StructField("city", StringType(), True),
    StructField("__corrupt_record", StringType(), True)
])

#read
df=spark.read.format("CSV").option("inferSchema", "true") \
    .schema(header) \
    .option("mode", "PERMISSIVE") \
    .option("columnNameOfCorruptRecord", "__corrupt_record") \
    .load("D:/emp.csv")
```



```
df.show()

df_valid=df.filter(df['__corrupt_record'].isNull()).drop("__corrupt_record")
df_valid.show()

df_invalid=df.filter(df['__corrupt_record'].isNotNull())
df_invalid.show()
```

Spark UDF :

User Defined Functions (UDFs) in Spark allow you to define custom functions to operate on DataFrame or Dataset columns. These functions can be written in various programming languages supported by Spark, such as Python, Scala, Java, or R. UDFs provide a way to extend Spark's built-in functions with your own logic, enabling complex data transformations and analyses.

Important Steps –

1. Define function
2. Register function
3. Use function

Registering UDFs: First, you define your custom function in your preferred programming language and register it as a UDF using the `udf()` method provided by `pyspark.sql.functions` (for Python). In Scala, you would use `udf` function from `org.apache.spark.sql.functions`.



Applying UDFs: Once registered, you can apply the UDF to DataFrame or Dataset columns using the withColumn() method (or select() method in Scala). This allows you to transform the data in the DataFrame by applying your custom function to one or more columns.

Input_data

country,year,temp
IND,2020,24
IND,2021,25
IND,2022,26
IND,2023,32
IND,2024,21
CAN,2020,-10
CAN,2021,-2
CAN,2022,10
CAN,2023,05
CAN,2024,-7

Ex –

```
from pyspark.sql import SparkSession
from pyspark.sql.types import *
from pyspark.sql.functions import *

spark=SparkSession.builder.appName("Without header").getOrCreate()
```



```
df=spark.read.format("CSV").option("header","true").option("inferSchema","true").load("D:/data.csv")
df.show()
df.printSchema()

#inbuilt
#df1=df.withColumn("country",lower(df["country"]))
#df1.show()

#UDF
def make_temp_zero(num):
    if num <= 0:
        return 0
    else:
        return num

#register function
make_temp_zero=udf(make_temp_zero,IntegerType())
#use
df2=df.withColumn("temp",make_temp_zero(df['temp']))
df2.show()
```



Set Log Level –

These log levels provide a structured way to categorize and prioritize log messages based on their severity and importance. By configuring the logging framework to output messages at different levels, developers can control the amount of information logged and focus on the most relevant details for troubleshooting and monitoring applications.

OFF: This level disables logging entirely. No logs will be emitted at this level.

INFO: This level is used for informational messages that describe the normal operation of an application. It provides general information about the application's execution.

WARN: This level indicates that there might be a potential issue or a non-critical problem. It's used to log warnings that don't necessarily cause the application to fail but may require attention.

ERROR: This level is used to log errors that occur during the execution of an application. These messages indicate significant problems that should be addressed. Errors may cause the application to fail or produce incorrect results.

DEBUG: This level is used for debugging purposes. It provides detailed information about the application's internal state and execution flow. Debug messages are typically used during development or troubleshooting to diagnose issues.

TRACE: TRACE level provides even more detailed information than DEBUG. It's often used to trace the flow of execution through the application, including method calls and variable values. TRACE logs are typically very verbose and are used for in-depth debugging.

FATAL: This level indicates a critical failure or error that results in the termination of the application. Fatal errors are severe issues that prevent the application from continuing normal operation. They often require immediate attention and may indicate serious problems with the system.

Use –



```
from pyspark.sql import SparkSession
from pyspark.sql.types import *
from pyspark.sql.functions import *

spark=SparkSession.builder.appName("Without header").getOrCreate()

spark.sparkContext.setLogLevel("FATAL")

df=spark.read.format("CSV").option("header","true").\

option("inferSchema","true").load("D:/data.csv1")

df.show()
df.printSchema()
```

RDD to DF :

We can convert RDD into DF to perform SQL like operation on it.

For that we need to follow below steps –

Define Your RDD: Start with an RDD. You can create an RDD from various data sources, such as by parallelizing a collection or by reading from a file.



Map the RDD to Row Objects: To convert the RDD to a DataFrame, you need to map each element of the RDD to a Row object. Row objects represent the structured data in each row of the DataFrame.

Create a DataFrame: Once you have an RDD of Row objects, you can create a DataFrame by using the toDF() method provided by the SparkSession.

Input Data –

getMethod:pull	192.168.10.11,www.facebook.com home 30min/profile
postMethod:post	192.168.10.10,www.instagram.com home 10min/profile
getMethod:pull	192.168.10.12,www.facebook.com home 1hr/home
postMethod:post	192.168.10.15,www.linkedin.com home 30min/profile

Exm –

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import *
from pyspark.sql import Row

spark=SparkSession.builder.appName("sample").getOrCreate()

rdd=spark.sparkContext.textFile("D:/web.logs")

rdd1=rdd.map(lambda x:x.replace(",","|")).map(lambda
x:x.replace(",","|")).map(lambda
```



```
x:x.replace(" ", "|").map(lambda
x:x.replace("/","|"))

def sep(line):
    ls=line.split("|")
    return ls[1],ls[2],ls[3],ls[4]

schema=Row("ip","visited_site","visite
d_page","duration")

df=rdd1.map(sep).map(lambda
x:schema(*x)).toDF()
df.show()
```

Broadcast join –

It is an optimization technique aimed at improving the performance of join operations by reducing data shuffling across the cluster.

a broadcast join is a type of join operation where one of the DataFrames involved in the join is small enough to fit entirely in memory on each worker node. This small DataFrame is then broadcasted to each worker node, avoiding the need to shuffle large amounts of data across the cluster during the join operation. Broadcast joins are especially useful



when one DataFrame is significantly smaller than the other DataFrame being joined.

1. Pure SQL Syntax –

```
df_res=spark.sql("select /*+ BROADCAST(dept) */ eid,ename,dname  
from emp inner join dept on emp.did=dept.did")
```

2. DF API syntax –

Perform a broadcast join

```
from pyspark.sql.functions import broadcast
```

```
joined_df = df1.join(broadcast(df2), 'id', 'inner')
```

The default broadcast threshold in PySpark is 10 MB. DataFrames smaller than or equal to 10 MB in size will be broadcasted by default. This threshold can be adjusted using the **spark.sql.autoBroadcastJoinThreshold** configuration parameter.

Autobroadcast join –

Auto broadcast join refers to the optimization where the framework automatically decides whether to broadcast smaller DataFrames for join operations based on the estimated size of the DataFrame. This is done to optimize the join process and improve performance by minimizing data shuffling across the cluster.

The decision to broadcast a DataFrame during a join operation is based on the size of the DataFrame and the broadcast threshold configured in



the Spark session. If the estimated size of a DataFrame is less than or equal to the broadcast threshold, PySpark will automatically broadcast it for the join operation.

You can enable auto broadcast join in PySpark by configuring the spark.sql.autoBroadcastJoinThreshold parameter.

Exp –

```
#pure SQL

from pyspark.sql import SparkSession

spark=SparkSession.builder.appName("Broadcast Join").getOrCreate()

df_emp=spark.read.format("CSV").option("header","true")\

.option("inferSchema","true").load("D:/emp.csv")

df_dept=spark.read.format("CSV").option("header","true")\

.option("inferSchema","true").load("D:/dept.csv")
```



```
df_emp.createOrReplaceTempView("emp")
df_dept.createOrReplaceTempView("dept")

df_res=spark.sql("select /*+
BROADCAST(dept) */ eid,ename,dname
from emp inner join dept on
emp.did=dept.did")

df_res.show()
```

```
#pure SQL

from pyspark.sql import SparkSession

spark=SparkSession.builder.appName("Br
oadcast Join").getOrCreate()

df_emp=spark.read.format("CSV").option
("header","true")\

.option("inferschema","true").load("D:
/emp.csv")

df_dept=spark.read.format("CSV").optio
n("header","true")\
```



```
.option("inferschema","true").load("D:/dept.csv")

df_emp.createOrReplaceTempView("emp")
df_dept.createOrReplaceTempView("dept")

df_res=spark.sql("select /*+
BROADCAST(dept) */ eid,ename,dname
from emp inner join dept on
emp.did=dept.did")

df_res.show()
```

Spark Submit command –

spark-submit \

- master <master-url> \
- deploy-mode <deploy-mode> \
- executor-memory <memory-per-executor> \
- executor-cores <cores-per-executor> \
- num-executors <number-of-executors> \
- conf <key>=<value> \
- py-files <comma-separated-python-files> \



```
<your-python-script>.py \  
<script-argument1> <script-argument2> ...
```

Here's what each parameter does:

--master: Specifies the URL of the Spark cluster manager. For example, spark://host:port, yarn, local, etc.

--deploy-mode: Specifies the deploy mode of the application, whether it's running in client mode or cluster mode.

--executor-memory: Sets the memory per executor (e.g., 1g, 2g, etc.).

--executor-cores: Sets the number of cores per executor.

--num-executors: Sets the number of executors.

--conf: Sets a Spark configuration property. You can use this to configure various Spark properties. For example, --conf spark.executor.instances=4.

--py-files: Specifies additional Python files to be included in the Python path. This is useful when your application depends on external Python modules.

<your-python-script>.py: The Python script you want to submit to Spark.

<script-argument1> <script-argument2> ...: Any arguments you want to pass to your Python script.

Exm –

```
spark-submit \  
  --master yarn \  
  --deploy-mode client \  
  <your-python-script>.py \  
  <script-argument1> <script-argument2> ...
```



```
--executor-memory 4g \  
--executor-cores 2 \  
--num-executors 4 \  
--conf spark.shuffle.service.enabled=true \  
--py-files my_dependencies.zip \  
my_spark_job.py \  
argument1 argument2
```

Module –

In Python, a module is a file containing Python definitions and statements. The file name is the module name with the suffix .py appended. Modules can contain functions, classes, and variables, and they allow you to organize your Python code into reusable units.

To use the code in a module, you can import it into another Python script using the import statement.

```
from module import some_function, SomeClass
```

Now you can directly use some_function and SomeClass without prefixing them with module.

```
some_function()
```

```
obj = SomeClass()
```



here we have used module.py file which having many functions written that we can reuse in our other file

ex - we have functions.py having many functions as below

In Python, a module is a file containing Python definitions and statements. The file name is the module name with the suffix .py appended. Modules can contain functions, classes, and variables, and they allow you to organize your Python code into reusable units.

To use the code in a module, you can import it into another Python script using the import statement.

```
from module import some_function, SomeClass
```

Now you can directly use some_function and SomeClass without prefixing them with module.

```
some_function()
```

```
obj = SomeClass()
```

here we have used module.py file which having many functions written that we can reuse in our other file

exp – we can use above declare function in other code



#program -

```
from pyspark.sql import SparkSession
```

```
from pyspark.sql.functions import *
```

```
from functions import *
```

```
spark=SparkSession.builder.appName("Broadcast Join").getOrCreate()
```

```
df_emp=read_csv(spark,'D:/emp.csv')
```

```
df_dept=read_csv(spark,'D:/dept.csv',delimiter='|')
```

```
df1=read_json(spark,'D:/sample.json')
```

```
df2=read_json(spark,'D:/sample_mul.json',multiline='true')
```

```
df_emp.show()
```

```
df_dept.show()
```

```
df1.show()
```

```
df2.show()
```

Broadcast Variables and Accumulators:

In the context of distributed computing frameworks like Apache Spark, "broadcast variables" and "accumulators" are special constructs used to efficiently share data and perform aggregate operations across distributed systems.



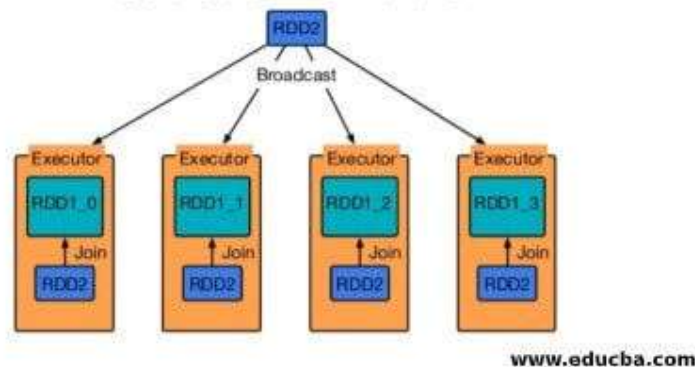
Broadcast Variables:

Broadcast variables are read-only shared variables that are cached and distributed to every worker node in a Spark cluster. They are used to efficiently distribute large, read-only datasets to all the nodes in the cluster.

Broadcast variables are useful when you have a large dataset that needs to be used across multiple parallel operations on worker nodes, but you want to avoid sending that data over the network multiple times.

Broadcast variables are created using the `broadcast()` method and can be accessed on worker nodes without being sent over the network multiple times.

Broadcast Variable



Accumulators:

Accumulators are variables that are only "added" to through an associative operation and are "local" to each worker node in a Spark cluster. They are mainly used for implementing counters or summing values during parallel operations.



Unlike broadcast variables, accumulators can only be "added" to and cannot be read by the driver program. Their values can only be accessed in a restricted manner to prevent race conditions.

Accumulators are particularly useful for tracking metrics or aggregating data during distributed computations.

Exm –

```
from pyspark.sql import SparkSession

# Create a SparkSession
spark = SparkSession.builder \

    .appName("BroadcastAndAccumulatorExample") \
    .getOrCreate()

# Create a broadcast variable
broadcast_data =
spark.sparkContext.broadcast(range(1,
1000))

# Create an accumulator
accum =
spark.sparkContext.accumulator(0)

# Define a function to process each
element
```



```
def process_element(x):
    global accum # Accessing the
    accumulator
    accum.add(x)
    return x *
sum(broadcast_data.value) # Using the
broadcast variable

# Create an RDD
rdd =
spark.sparkContext.parallelize(range(1
0))

# Apply the processing function to
each element of the RDD
processed_rdd =
rdd.map(process_element)

# Trigger an action to execute the
transformations and collect the result
result = processed_rdd.collect()

# Print the result and the value of
the accumulator
print("Result:", result)
print("Accumulator Value:",
accum.value)
```



```
# Stop the SparkSession  
spark.stop()
```

Data Skewness –

imbalance in the distribution of data across partitions, which can lead to performance issues and suboptimal resource utilization during data processing.

When a dataset is skewed, one or a few partitions contain significantly more data than others, causing those partitions to take much longer to process than the rest.

This can result in a situation where some worker nodes are idle while others are overloaded, leading to a slowdown in overall processing time.

Data skewness is a common problem in distributed data processing frameworks like Spark because they rely on parallelism to achieve high performance. When the workload is not evenly distributed across partitions, the efficiency of parallel processing diminishes, impacting the overall job performance.

#causes :

1.Hot Keys: :When a particular key in a dataset appears significantly more frequently than others, it can cause data skewness. This is often seen in scenarios like user IDs, where a few users generate much more data than the rest.

2.Data Modification Operations: Certain operations like filtering, grouping, or aggregating can exacerbate skewness if they result in a highly uneven distribution of data.



3.Data Skew from Aggregations: Aggregations that group data by a skewed key can lead to skewed partitions. This is especially true if the aggregation keys have very different cardinalities.

#impact

Uneven Resource Utilization

Slow Processing

Increased Latency:

Job Failures:

Inefficient Shuffle Operations:

Resource Wastage:

Unpredictable Performance:

Complex Debugging and Tuning:

#technique to avoid it ?

1.repartition

2.Skew-Aware Operations: Some operations in Spark, like skewedJoin, are designed to handle skewed data more efficiently. These operations identify skewed keys and handle them differently, often using more resources for processing the skewed partitions.

3.Salting: Salting involves adding a random or semi-random prefix to the data, which effectively distributes skewed data more evenly across partitions. This can be particularly useful when the skew is caused by certain key values dominating the dataset.



Steps for Salting -

add salted factor

group by

remove salted factor

Exp –

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col,
concat, lit, rand

# Create a SparkSession
spark = SparkSession.builder \
    .appName("DataSkewnessExample") \
    .getOrCreate()

# Read data from CSV file
df =
spark.read.format("CSV").option("header", "true").load("input.csv")

# Define the number of salt partitions
num_salts = 10

# Add salt to mitigate skewness
salted_df = df.withColumn("salt",
(rand() *
```



```
num_salts).cast("int").cast("string"))
salted_df =
salted_df.withColumn("salted_key",
concat(col("key"), lit("_"),
col("salt")))

# Group by salted_key to distribute
data more evenly
result =
salted_df.groupBy("salted_key").count(
).orderBy("salted_key")

# Show the result
result.show()

# Stop the SparkSession
spark.stop()
```

Spark Memory Management –

In Apache Spark, memory management is crucial for optimizing performance and avoiding out-of-memory errors, especially when dealing with large datasets. Spark manages memory in two main areas: execution memory and storage memory.

Driver Memory: The driver is the process where the SparkContext lives, and it coordinates the execution of the Spark application. Driver memory is the amount of



memory allocated to the Spark driver process. This memory is used for storing SparkContext, SparkSession, and for coordinating tasks, such as creating execution plans, distributing tasks to executors, and aggregating results. Driver memory typically houses data structures like DAGs (Directed Acyclic Graphs), task schedules, and other metadata.

You can set the driver memory using the `--driver-memory` flag when submitting your Spark application via `spark-submit` or by setting `spark.driver.memory` in your Spark

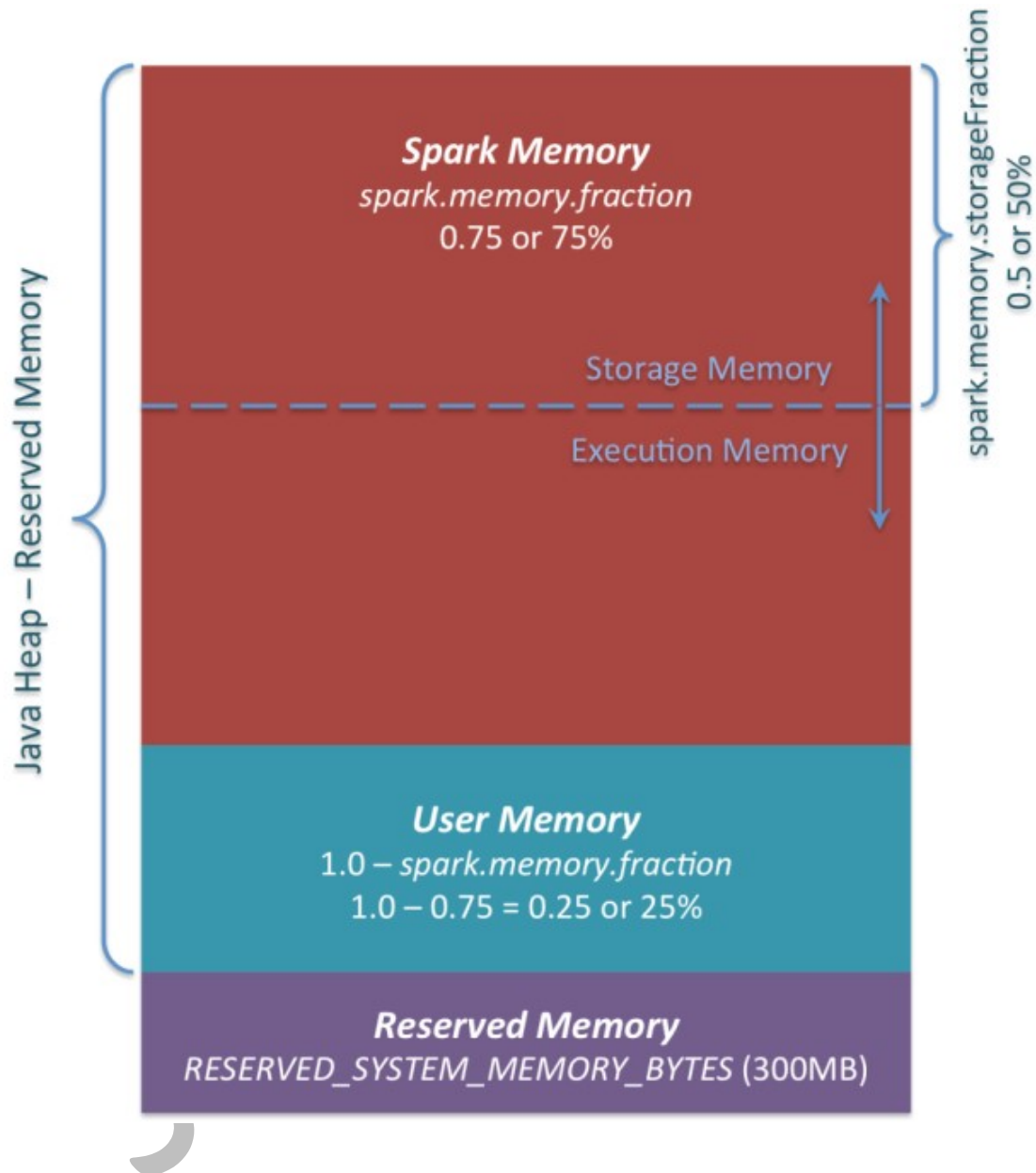
```
spark-submit --driver-memory 4g ...
```

Executor Memory: Executors are worker processes responsible for executing tasks and storing data for Spark applications. Executor memory is the amount of memory allocated to each executor. This memory is used for caching data, shuffling data during operations like joins or aggregations, and performing computations on the data.

You can set the executor memory using the `--executor-memory` flag when submitting your Spark application via `spark-submit` or by setting `spark.executor.memory` in your Spark configuration. For example:

```
spark-submit --executor-memory 8g ...
```

Executor Memory Further classified in –



Reserved Memory: Reserved memory is the memory reserved by Spark for system overheads and internal metadata structures. This includes memory required for JVM overhead, off-heap memory, and other internal bookkeeping. It's important to reserve a certain amount of memory to ensure the stability and proper functioning of the Spark application.



User Memory: User memory is the memory allocated for user-defined data structures and objects within the Spark application. This includes memory used for storing RDDs, DataFrames, Datasets, and other user-created data structures. User memory is used for caching data, storing intermediate results, and holding user-defined variables and objects.

Spark Storage Memory: Spark storage memory is a portion of the reserved memory allocated specifically for caching data. When you cache RDDs, DataFrames, or Datasets using Spark's caching mechanism, the data is stored in the Spark storage memory. This memory is used to hold frequently accessed data in memory to improve performance by reducing the need for recomputation.

Spark Execution Memory: Spark execution memory is the portion of the reserved memory allocated for storing data structures and intermediate results during task execution. This includes memory used for shuffles, joins, sorts, aggregations, and other computational tasks performed by Spark. Spark execution memory is used by the executor processes to perform computations on the data and to store temporary data structures needed for task execution.

Configuring these memory components properly is essential for optimizing the performance and stability of your Spark applications. You can adjust the memory settings using Spark configuration properties such as `spark.memory.fraction`, `spark.memory.storageFraction`, `spark.memory.offHeap.size`, etc. By tuning these properties based on your application's requirements and the available resources in your cluster, you can ensure efficient memory utilization and avoid out-of-memory errors.

Dynamic Resource Allocation –

Dynamic resource allocation in Apache Spark refers to the ability of the Spark cluster manager to adjust the allocation of resources (such as CPU cores and memory) among applications dynamically based on their resource needs. This feature helps in maximizing resource utilization and improving overall cluster efficiency.



Here's how dynamic resource allocation typically works in Spark:

Initial Allocation: When a Spark application is submitted to the cluster, it initially requests a certain amount of resources (e.g., number of CPU cores and amount of memory) based on the configuration settings or the resources specified by the user.

Execution: As the application starts executing, it may utilize resources unevenly over time due to factors like varying workload characteristics, data skew, or stages with different resource requirements.

Resource Adjustment: Spark's dynamic resource allocation monitors the resource usage of running applications. If an application is not fully utilizing its allocated resources or if there are idle resources in the cluster, the resource manager can reallocate those resources to other applications that need them more.

Scaling: Similarly, if an application requires additional resources beyond its initial allocation (e.g., due to increased workload or data processing needs), the resource manager can allocate more resources dynamically to accommodate the growing demand.

Deallocation: Conversely, if an application finishes its tasks or releases resources it no longer needs, those resources can be returned to the resource pool and made available for other applications.

Dynamic resource allocation in Spark helps in achieving better resource utilization, improving overall cluster throughput, and providing better support for multi-tenancy environments where multiple applications share the same cluster resources.

Parameter which we need to configure -

`spark-submit \`



```
--master <master-url> \  
--deploy-mode <deploy-mode> \  
--executor-memory <memory> \  
--executor-cores <cores> \  
--num-executors <count> \  
--driver-memory <memory> \  
--name <name> \  
--class <main-class> \  
--conf spark.dynamicAllocation.enabled=true \  
--conf spark.dynamicAllocation.initialExecutors=<initial-executors> \  
--conf spark.dynamicAllocation.minExecutors=<min-executors> \  
--conf spark.dynamicAllocation.maxExecutors=<max-executors> \  
<application-jar> <application-arguments>
```

Catalyst Optimizer –

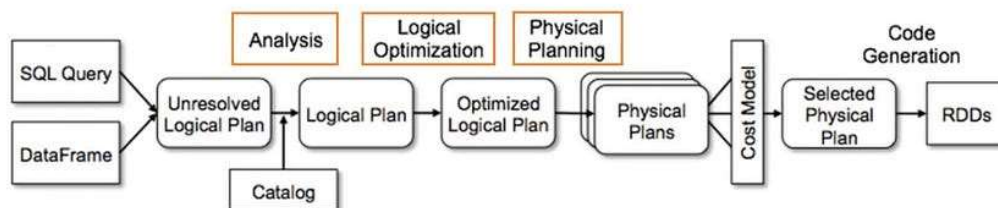
At the core of Spark SQL is the Catalyst optimizer, which leverages advanced programming language features (e.g. Scala's pattern matching and quasi quotes) in a novel way to build an extensible query optimizer. Catalyst is based on functional programming constructs in Scala and designed with these key two purposes:

Easily add new optimization techniques and features to Spark SQL

Enable external developers to extend the optimizer (e.g. adding data source specific rules, support for new data types, etc.)



Catalyst Optimizer



How it works

The functioning of Catalyst Optimizer can be segmented into four primary phases:

Parsing: In this phase, Catalyst converts SQL queries into an abstract syntax tree (AST), which represents the structure of the query in a way that is easier for the computer to understand.

Analysis: Next, Catalyst verifies the AST for semantic correctness. It resolves attributes by looking at catalog information and also resolves functions. The output of this phase is a “resolved logical plan.”

Logical Optimization: Here, the Catalyst Optimizer applies a series of rule-based optimizations to the resolved logical plan. Some standard techniques include predicate pushdown, projection pruning, and Boolean simplifications.

Physical Planning: The logical plan is now converted into one or more physical plans. Then, Catalyst selects the most optimal physical plan using cost-based optimization, where the cost of each plan is estimated based on the sizes of its inputs.

Code Generation: Finally, Catalyst generates JVM bytecode to run the selected physical plan. This stage uses ‘whole-stage code generation’, which compiles



multiple operators into a single function, eliminating the overhead of invoking multiple functions.

Its by default enabled In spark

#optimization

#Common issues in spark in production